RICE UNIVERSITY

OCCA: A Unified Approach to Multi-Threading Languages

by

David Medina

A Thesis Submitted
in Partial Fulfillment of the
Requirements for the Degree

Master of Arts

Approved, Thesis Committee:

Timothy Warburton, Chair
Associate Professor of Computational and Applied Mathematics

Beatrice Riviere
Professor of Computational and Applied Mathematics

William W. Symes
Noah Harding Professor of Computational and Applied Mathematics

---

William W. Symes
Noah Harding Professor of Computational and Applied Mathematics

Houston, Texas

April, 2014

ABSTRACT

OCCA: A Unified Approach to Multi-Threading Languages

by

David Medina

With the current trend of using co-processors for accelerating computations, we are presented with architectures and corresponding programming languages. The inability to predict lasting languages and architectures has led to the development of distinct languages and standards. This thesis details my work on OCCA, a unified threading language presented as a portable solution to hardware-accelerated coding that combines aspects of OpenMP, OpenCL, and CUDA. With the similarities between OpenMP, OpenCL and CUDA, I present a macro-based approach on a unified kernel language that currently encompasses OpenMP, OpenCL and CUDA. Along with kernel generation, OCCA includes an API (application programming interface) which serves as a wrapper on the three multi-threading languages. The back-end on OCCA dynamically compiles and loads function objects for a flexible run-time environment to use different hardware architectures.

Computational results using a spectrum of methods, namely finite difference, spectral element and discontinuous Galerkin methods, utilizing OCCA are shown to deliver portable high performance on different architectures and platforms. The finite difference method chapter reverse engineers optimized code written in CUDA and used in industry, discusses distinct features available in CUDA and compares OCCA im-

plementations using different optimization techniques. The spectral element method and discontinuous Galerkin methods are derived from two projects I worked on during my studies: gNek, a distributed high-order spectral element method (SEM) implementation for the incompressible Navier-Stokes equations, and RiDG, equipped with discontinuous Galerkin (DG) to simulate acoustic wave equations under different assumptions in the material anisotropies. The parallel algorithms used to achieve high parallelization for GPU acceleration are discussed in both, gNek and RiDG, together with performance results.

Contents

Abstract ii

1 Introduction 2

1.1 High Performance Computing 3

1.2 GPU Programming 4

1.3 Compiler Constructs 5

1.4 Language Interface 6

1.5 Thesis Outline 8

2 Multi-Threading Architectures 11

2.1 Central Processing Units 11

2.1.1 Caching 12

2.1.2 Vectorization 13

2.1.3 Multi-Threading 14

2.2 Graphics Processing Units 16

2.2.1 Introduction 16

2.2.1.1 Hardware and Programming Model 17

2.2.2 Memory Types 18

2.2.2.1 Global Memory 19

2.2.2.2 Shared Memory 19

2.2.2.3 Private Memory 20

2.2.3 Optimization Techniques 20

2.2.3.1 Work-groups and Work-items 20

2.2.3.2 Memory 22
2.3 Architecture Similarities 26
2.3.1 Memory Fetching 27
2.3.2 Vectorization 27
2.3.3 Multi-threading 28

3 OCCA **29**
3.1 OCCA Host API 30
3.1.1 OCCA device 32
3.1.2 OCCA memory 32
3.1.3 OCCA kernel 33
3.2 OCCA Device API 33
3.2.1 Macro-based kernel generation 34
3.2.2 Handling multi-threading architectures 35
3.2.3 Threaded barriers 37
3.2.4 Registers and register arrays 38
3.3 Finite Difference Example 40
3.3.1 Governing Equations 41
3.3.2 Discretization 41
3.3.3 Finite Difference Performance 44

4 Vertical Transverse Isotropy - Finite Difference **46**
4.1 Wave Equation Discretizations 47
4.2 Reverse Engineering CUDA Optimizations 49
4.3 OCCA Kernel Tuning 56
4.3.1 Hinting Constant Pipeline 57
4.3.2 Performance Comparisons 58

5 gNek **60**
5.1 Introduction 61

5.2 Discretization 62
5.2.1 Variational Form 63
5.2.2 Test and Trial Spaces 64
5.2.3 Discretized Operators 67
5.3 Implementation 70
5.3.1 Algorithm Walkthrough 70
5.3.2 Memory Layout 74
5.4 Performance Results 77

6 RiDG 80
6.1 Introduction 80
6.1.1 Acoustic Wave Equation 81
6.1.2 Vertical Transverse Isotropy 82
6.1.3 Tilted Transverse Isotropy 84
6.2 Discretization 85
6.3 Implementation 87
6.4 Performance Results 90

7 Conclusions and Future Work 93
7.1 Conclusion 93
7.2 Future Work 95

A Appendix: OCCA Kernel Keywords 96

B Appendix: Code 101
B.1 gNek: OCCA Stiffness Operator Application Kernel 101
B.2 RiDG: OCCA Volume Term Application Kernel 105
B.3 RiDG: OCCA Surface Term Application Kernel 107

References 112

CHAPTER 1

Introduction

Initially, performance growth was linearly correlated with transistor growth which followed Moore's Law [42]. Once power consumption became a bottleneck, Dennard scaling allowed processors to maintain their transistor scaling alongside constant power consumption [11]. Together with transition growth and Dennard scaling, there were architectural changes that allowed for the peak performance to continue scaling, even after Dennard scaling reached its limit [10]; changes such as caching, vectorization and the addition of multiple cores. Along with the adapting architecture, memory bandwidth became a large bottleneck and cache was introduced, followed by vectorization instructions to make use of memory fetching programming practices. Co-processors such as the original floating point unit, and more recently the graphics processing units, became essential to accelerating programs. Throughout this trend of architectural changes, many alternative architectures and standards were both, created and deemed obsolete. The goal of the thesis was to find similarities in current

architectures and languages, and create a flexible and extensible framework to unify them.

1.1 High Performance Computing

Until 2001, the top supercomputers were built on a parallel architecture involving multiple single-core vector machines. Multi-core processors then became the standard with co-processor acceleration as early as 2005 [1]. Not knowing which architectures will stand the test of time places programmers in an awkward position. For instance, IBM's cell architecture, known for as the graphics accelerator choice in the Playstation 3, was used in IBM's Roadrunner, the first petaflop-capable machine. Although the cell architecture had high promise, it was put on hiatus less than a decade after development [9]. Graphical processing units (GPUs) quickly dominated the co-processor competition with the only current competitor being Intel's Xeon Phi. Although few supercomputers make use of the Xeon Phi as an accelerator, 4 out of the 6 systems listed in the top500 are currently the top 10 in performance, including the top supercomputer in the list [1]. Chapter 2 outlines main advances in traditional CPU architectures from cache usage, vectorization and ultimately leading to the use of multiple independent cores. Along with the optimizations found in the CPU architecture, chapter 2 goes through the GPU architecture, its optimizations and the similarity between best programming practices for CPUs and GPUs.

1.2 GPU Programming

Initially, instructions were passed from the CPU to the GPU with the use of programmable graphics shaders, mainly using OpenGL's shading language (GLSL), DirectX's high-level shader language (HLSL), or NVIDIA's Cg shading language. As early as 2004, shaders began to be used for general purpose computations outside of graphics. The Brook language was developed to reduce the complexity of using graphics shaders for general purpose computations [8]. Brook was a predecessor to NVIDIA's CUDA, extending the C language to include GPU variable and function declarations to hide shader-programming complexity. NVIDIA released CUDA in 2007 and extended the C++ language to allow GPU programming in a similar manner to the Brook language. Adopting CUDA for development vendor-locks the implementation to NVIDIA GPUs for hardware acceleration [6].

The Khronos group, a consortium focused on creating open standards, released the OpenCL standard for programming heterogeneous platforms including CPUs, GPUs, Intel's Xeon Phi and field-programmable gate arrays (FPGAs). A year after CUDA's release, NVIDIA and Advanced Micro Devices (AMD) gave their first functional OpenCL demo in Siggraph Asia 2008 [40]. Implementations of the OpenCL standard, however, provide a C library API rather than a language extension.

With the current trend of using co-processors as accelerators, a zoo of devices and languages to support them have been created and announced. Figure 1.1 shows the

languages and standards that are discussed throughout chapter 3 and the available architectures each language can target.

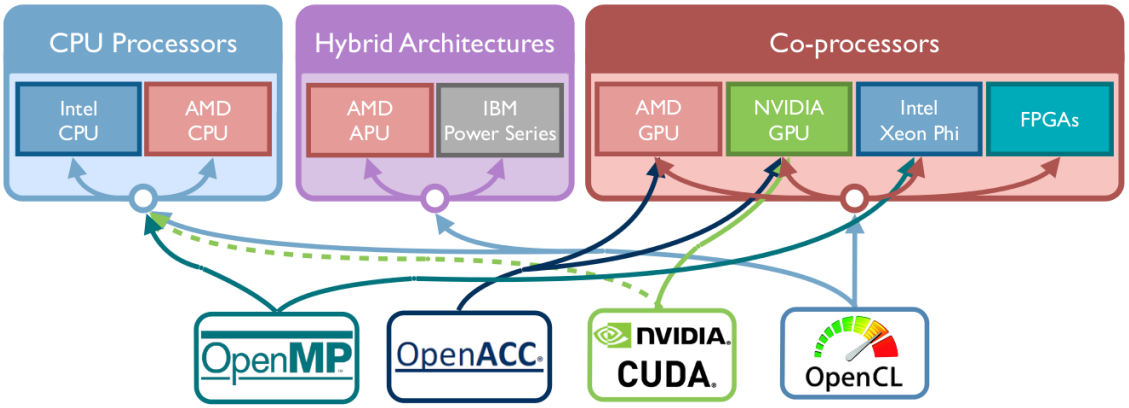


Figure 1.1 : This diagram shows the OpenMP, OpenACC and OpenCL standards for programming different architectures as well as CUDA, a proprietary language used to program NVIDIA GPUs.

1.3 Compiler Constructs

New standards, languages and extensions often accompany the arrival of new architectures, such as OpenMP for multi-core processors and both CUDA and OpenCL for GPUs. Refactoring legacy codes to suit new architectures with minimal adjustments has been a recurring challenge, inspiring standards such as OpenMP and OpenACC [14].

OpenMP and OpenACC standards focus on giving hints to compilers to perform specific optimizations for parallelizing code regions with the use of pragmas (optional directives). Using supported compilers, code regions can be parallelized to use thread-

based parallelization (OpenMP) or GPU acceleration (OpenACC) with minimal code modification. Although compiler-aided parallelization is achieved with pragma insertions, there are several obstructions to automating parallelization. The use of OpenMP requires the use of only thread-safe functions and knowledge about memory residency for variables to prevent read-write conflicts. Similarly, OpenACC has memory residence ambiguity from pointer aliasing, often requiring memory transfers between the host and device before and after critical regions.

1.4 Language Interface

Apart from compiler-based solutions for refactoring, we introduce different approaches to platform flexibility on GPU-accelerated projects. As previously mentioned, CUDA works only on NVIDIA cards; and as a result, there are many projects are aimed on automating translations between CUDA and OpenCL for device-portability. Aside from using compiler pragmas for code generation, we discuss projects that create translation layers between languages using code translation or a unifying API. Swan [29] and CU2CL [35] were two of the first of few projects to attempt a conversion between CUDA and OpenCL, both using distinct source-to-source methods.

Swan is a perl-based library that uses regular expressions to translate CUDA kernels to OpenCL kernels. From AMD's "Porting to OpenCL" [12] website, it is clear that most of the CUDA to OpenCL kernel translations can be done with string replacements.

Swan, like many approaches for enabling portability [3], also provided an API that shadowed CUDA and OpenCL runtime API to hide the difference in API between the two languages.

On the other hand, CU2CL used clang's preprocessor, abstract syntax trees and framework to facilitate CUDA to OpenCL code translation [35]. The novelty in CU2CL is the ability to translate both kernels and source code, while the approach by Swan introduces an API layer to the source code.

GPU Ocelot is another project towards programming heterogeneous platforms. Led by a group in Georgia Tech's Computer Architecture and Systems Laboratory (CASL), Ocelot is capable of transforming PTX (NVIDIA's intermediate parallel assembly language) to low level assembly languages in different platforms. Targeted platforms include the CPU architecture through emulated kernels and AMD GPUs through a transformation to AMD's CAL assembly language [13]. The diagram figure 1.2 shows an overview of the different portability approaches discussed in this section.

So far, we discussed projects aimed on platform portability; we now introduce some portable libraries focused on mathematical functionalities rather than general purpose computations. ViennaCL [41] is an example of a library with linear algebra routines on multiple platforms. ViennaCL is able to switch enable OpenMP, OpenCL and CUDA for parallel computations at compile-time and choose the back-end platform for executing kernels. PyFR [46] is a python library focused on numerical solver im-

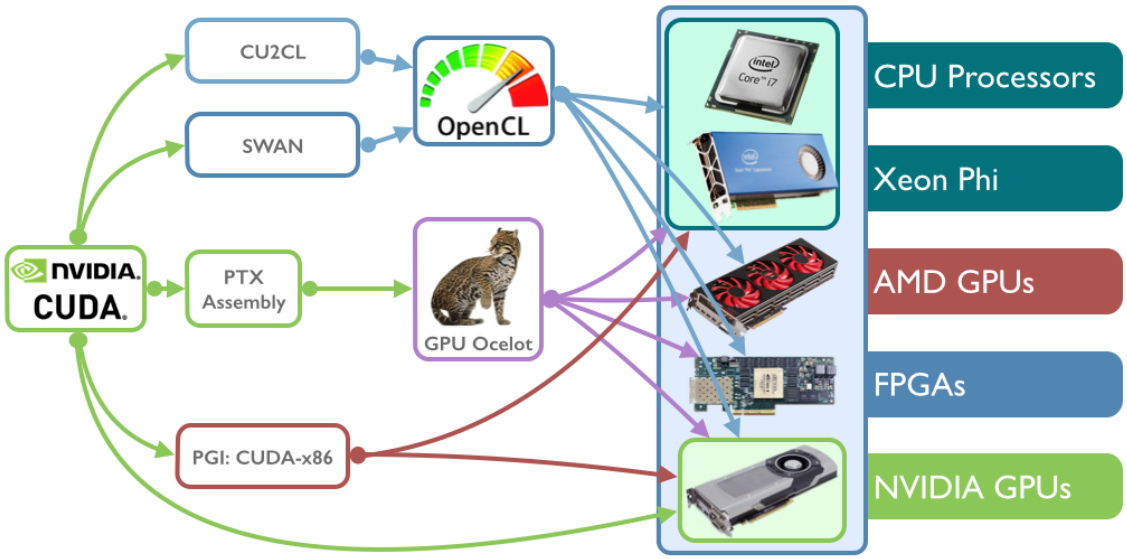


Figure 1.2 : The diagram shows the different approaches for adding portability to CUDA mentioned in this section.

plementations that is able to use either OpenMP, OpenCL and CUDA for advection-diffusion type equations. However, the kernel generations available in PyFR are fixed to their set of numerical routines available, rather than offering a generic API to code for the different platforms.

1.5 Thesis Outline

With the rapid changes in technology and growing ways to operate it, it is of interest to develop an approach that is not biased to a single programming language. The goal for this thesis was to develop a tool that addressed the problems that occurred with the introduction of new architectures and programming languages. A majority

of the work found in the thesis came from my experience in porting multiple projects between OpenMP, OpenCL and CUDA languages. The monotonous and repetitive nature of translating across languages became a large influence in the development of OCCA.

In order to find commonalities between architectures, chapter 2 first explores best common practices in the two common architectures found in high performance computing: traditional multi-core processors and many-core graphical processing units. The information about the architectures in chapter 2 sets the background for the approach taken in OCCA. Chapter 3 outlines the main contribution of this thesis, OCCA, a library which includes an API that abstracts back-ends and kernel languages from OpenMP, OpenCL and CUDA. Although OCCA unites different threading platform back-ends, the main contributions is the abstraction of the kernel language. The different projects mentioned in the previous section focused on creating some mapping between two or more programming languages for these accelerators, or switching between multiple platforms for computation purposes. Using a macro-based approach, an OCCA kernel can be expanded at runtime to suit OpenMP with dynamic pragma insertions or a device kernel using either OpenCL or CUDA and be used to streamline the incorporation of future languages with ease.

Three applications that utilize OCCA are given in the following three chapters to display the flexibility and performance available through OCCA. To give a broad spectrum of examples, I chose to give a finite difference method, spectral element

method, and a discontinuous Galerkin method example. The first numerical example is given in chapter 4, where I go through a finite difference implementation of the acoustic wave equation with some anisotropy. The second numerical method example found in chapter 5 uses a hexahedral spectral element method to solve for numerical solutions for the 3D Navier-Stokes equations. The last numerical example found in chapter 6 uses a discontinuous Galerkin method for the acoustic wave equation under different velocity models.

CHAPTER 2

Multi-Threading Architectures

In order to understand the algorithms proposed, knowledge of targeted architectures is crucial and similarly, current trends are good predictors for future architectures. The more widely known CPU optimizations are first explained to smooth the transition into understanding GPU optimizations. Following the architectural backgrounds, the similarities in the optimization techniques discussed throughout the chapter are exposed. The resemblance of the two architectures then become the basis of the development of OCCA which introduces chapter 3.

2.1 Central Processing Units

Initially, central processing units, more commonly refererred to as CPUs, were made up of a single scheduler and instruction execution pipeline. Depending on transistor growth for computational speed, processor manufacturers saw a linear correlation

between transistors and performance. However, because power consumption scaled with the transistors, a power bottleneck became apparent. Using Dennard scaling, manufacturers exploited the fact that shrinking a transistor by some constant size would also reduce the required voltage and current to power such transistor by the same scaling factor [16].

The brute force approach for increasing processor performance started suffering from diminishing returns around the year 2001 [10]. In this section, I discuss the different architectural changes CPUs underwent to maintain peak performance scaling.

2.1.1 Caching

It became apparent that memory bandwidth was becoming a large bottleneck and even to date, multiple layers of bandwidth speeds are the bottleneck in high performance computing. As early as 1989, Intel began experimenting with the caching of data on the 80486 processor which, by definition, is the action of storing away resources. Much of the CPU architectural optimizations in the design process has dealt with certain programming assumptions. In terms of caching, when data is queried from an address in DRAM, the processor presumes local memory usage and thus stores neighboring addresses onto on-chip memory as seen in figure 2.1. Over time, many levels of cache have been introduced to the point where it is common for consumer processors to have L1, L2, and L3 cache.

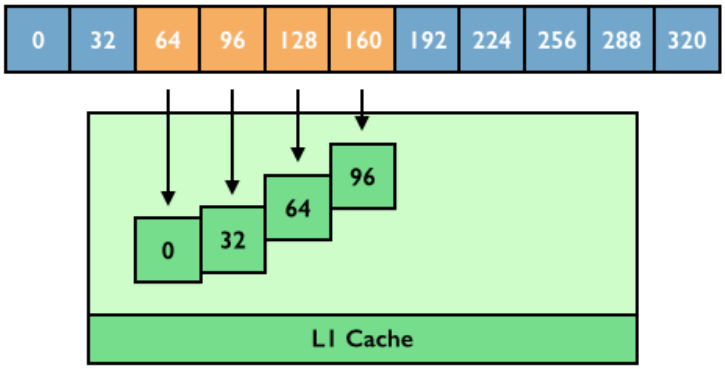


Figure 2.1 : When data is accessed from DRAM, neighboring data is sent and stored through the different cache levels.

2.1.2 Vectorization

With the addition of cache, programming practice encouraged the use of contiguous memory which could correspond to using the same operations on local data. To make use of this programming practice, Intel introduced in 1998 an extension to their instruction set that allowed for SIMD (single instruction, multiple data) operations [31]. Vectorization, the use of the SIMD intrinsic calls, could be used alongside caching to reduce bandwidth overhead and execute multiple instructions using large registers (figure 2.2).

Intel's MMX technology became the first instruction set for the x86 architecture capable of SIMD operations. MMX instructions allowed for a SIMD instruction to use two floating point units in cycle. The vectorization size, or SIMD width, is commonly associated with the number of floating point operations available in one

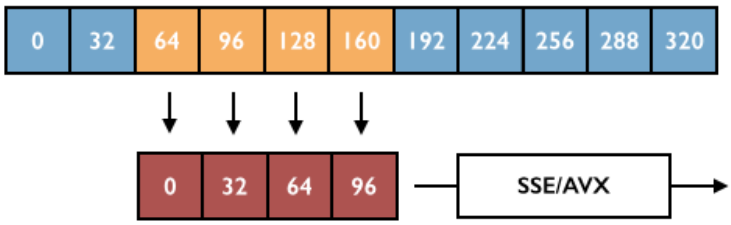


Figure 2.2 : Using large registers, vectorization instructions can apply multiple operations in one cycle.

instruction; for MMX, the SIMD width would be two.

Following the MMX instruction set, Intel released the streaming SIMD extensions, generally called by its abbreviation SSE. The SSE-enabled processors had a SIMD width of four, twice the operations offered by MMX. With the availability of large registers, more general operations were featured in the later SSE2, SSE3, and SSE4 instruction extensions.

Subsequently, the advanced vector extensions (AVX) instruction set was released for the 2011 Sandy Bridge processors that had a SIMD width of eight, supporting 256-bit registers. With the new Intel Xeon Phi co-processor, an extended AVX (EAVX) instruction set was released to allow the Xeon Phi to support 512-bit registers, giving it a SIMD width of 16.

2.1.3 Multi-Threading

While processor manufacturers continued to improve their architectures, the large and rapid calculations computers made available quickly became prominent across

different fields of study. Many problems, such as those found in seismic exploration, required a vast amount of storage. Parallel computing, through the use of connecting multiple computing units, became the viable solution for computing large problems. Appending computing units resulted in a linear increase of available memory storage and allowed for each compute unit to work independent to other units.

In 1994, the Center of Excellence in Space Data and Information Sciences (CESDIS) built a network of processors, resulting what is known as one of the first Beowulf clusters; a term coined to describe a combination of consumer-level computers [4]. The Beowulf cluster developed into a cheap supercomputer that would allow for parallel computations. Akin to architecture changes focusing on increasing computation performance at the finest level of computation, creating an efficient network of computing units focused on a coarser level of computation.

Seeing the progression of high performance computing, processor manufacturers began to work on multi-core processors which would innately incorporate multiple processing units in one processor. In 2001, IBM became the first company to release a microprocessor with a dual-processor, quickly followed by Intel with the Celeron line of processors, a consumer-level chip available with two processing cores [34]. During the last decade, consumer-level processors have been able to posses up to eight cores while server-grade processors can be found with more than 10 processing cores.

2.2 Graphics Processing Units

Originally, floating point units (FPU) were co-processors in which the CPU would offload specialized instructions for the FPU to compute. Similarly, graphical processing units (GPU) were specifically designed for the computations needed in 3D rendering and have followed the same offload model as the original FPUs.

Although the GPU programming model differs from that found in traditional processors, the architecture resembles a prospective CPU architecture. For example, an alternative to current integrated graphics is the accelerated processing units (APU) currently developed by AMD that handles GPU technology as an essential part of the future processors. In this section, I discuss the optimizations used for GPU architectures and reveal similarities between the GPU and CPU architectures.

2.2.1 Introduction

Before mentioning optimization techniques for GPU, an introduction to the GPU architecture is given. First, the hardware and programming model are described, followed by the different memory types found in the hardware and their use in the programming model.

2.2.1.1 Hardware and Programming Model

To understand the GPU architecture, I first discuss its history and explain the processes that led to the current architecture. Following the history of the GPU, I review the programming model and its relation with the hardware.

GPUs were created to expose the natural parallelism found in a 3D scene rendering process. Specifically, the stage of the graphics pipeline requiring such parallelism is in the update of each fragment (similar to a pixel) which contained attributes such as a surface normal, light intensity and material properties. A set of 3D objects are assigned to a specific shader (graphics compute kernel) which calculated the color of each fragment the objects generated by using the same computational kernel. However, to render the diverse materials available in real-life, the ability to quickly change shaders was required.

To exploit the high parallel nature of algorithms found in 3D rendering, large number of FPUs are found on GPU architectures, often referred to as streaming cores. Arrays of FPUs are comparable to vectorization units found in CPU architectures. However, to achieve the flexibility required in 3D graphics applications, arrays of 16, 32 or 64 FPUs (warps in CUDA terminology and wavefronts in AMD's terminology) were given their own instruction scheduler.

Although knowledge of the hardware is crucial for optimizing GPU kernels, the programming model hides most hardware features while integrating them through best-

practices found in GPU coding. For example, the work-group and work-item programming model seen in figure 2.3 will make use of the warp/wavefront hardware if the programmer uses multiples of 16 as work-group sizes. An assortment of optimizations found in GPU programming is given in section 2.2.3.

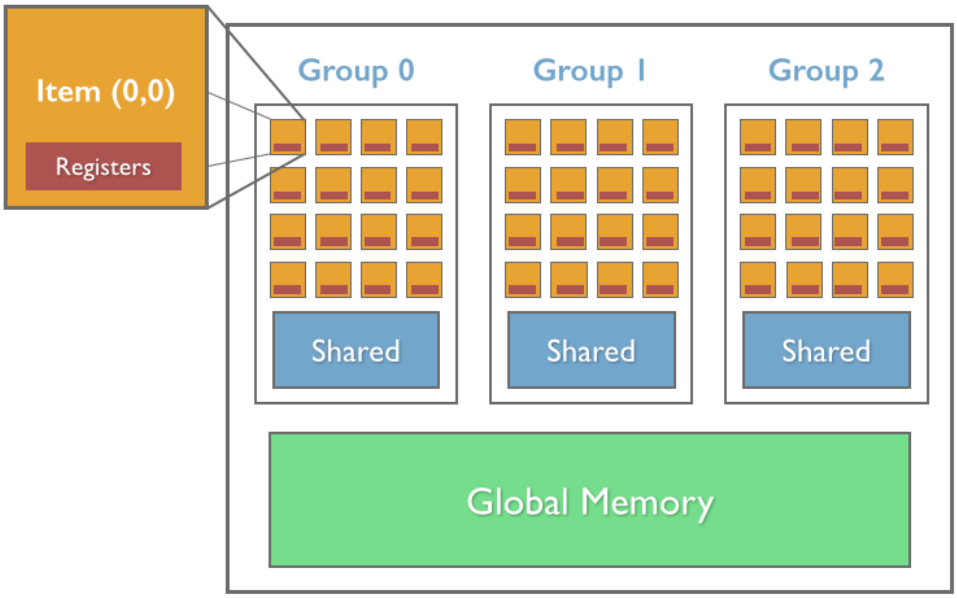


Figure 2.3 : GPU Hierarchy showing work-group and work-item relation as well as the different memory types and their scopes.

2.2.2 Memory Types

There are three fundamental memory types found on the GPU architecture: global, private, and shared memory. Figure 2.3 shows the work-group and work-item hierarchy alongside the memory layout.

In the nested memory model, shared memory follows global memory, and is available

to all work-items in the same work-group. Lastly, private memory is scope-specific for a single work-item, usually making use of registers.

2.2.2.1 Global Memory

Global memory refers to the majority of data stored in GPU DRAM. Akin to the CPU DRAM, any streaming processor in the GPU is able to access data stored in global memory. A second similarity with CPU DRAM is the high latency in retrieving data from global memory.

Although GPUs has high DRAM peak bandwidth, the memory channels are shared by all streaming processors and thereby making memory fetching a large bottleneck in the GPU pipeline. To reduce the effects of the high latency in global memory fetches, intermediate memory layers were added to the GPU architecture; a similar approach taken by CPU processors with the same underlying technology.

2.2.2.2 Shared Memory

The second type of memory I will discuss is shared memory which gives the flexibility of granting work-items in a work-group concurrent access without a large performance sacrifice. Shared memory is usually used for data that is spatially-dependent, such as finite difference stencil data, or an array of constants, such as finite difference stencil weights, that can be re-used throughout a work-group.

Due to shared memory residing in L1 cache, the access of shared memory is relatively fast when compared to accesses from global memory. Rather than the automatic caching found in CPU processors, shared memory allows the programmer to manually cache data.

2.2.2.3 Private Memory

The last type of memory I will discuss is private memory which is often regarded as the fastest memory type. Using registers, private memory is exclusive to each work-item and is usually the storage for intermediate computations.

2.2.3 Optimization Techniques

With the introduction of the GPU infrastructure, I will list principal optimization techniques often used in GPU best-practice programming. I go though the different memory types and give reasons of use together with their disadvantages.

2.2.3.1 Work-groups and Work-items

In this section, I mention hardware-specific optimizations related to work-group arrangements and work-item instructions. For prior information on the hardware, the reader is asked to see section 2.2.1.

Each work-group is mapped to hardware in a streaming multiprocessor (SM). Due

to the granularity of the scheduler, a work-group constitutes of a group of warps/wavefronts; therefore, the number of streaming cores retained by a work-group is a multiple of streaming cores found in a warp/wavefront for the given architecture of the GPU. For example, a work-group of size 33 might require 48 or 64 cores to be reserved for the kernel execution where the supplementary 15 or 31 streaming cores always execute NOPs (no-operation instructions). By virtualizing work-groups, the GPU is able to handle work-group counts magnitudes greater than the hardware is able to support by launching work-groups as hardware becomes available. In other words, after a work-group finishes with its computations, the GPU is able to schedule another work-group to the same group of warps/wavefronts. Another hardware optimization making use of this feature is the ability to context-switch across work-groups when waiting for data from global memory. With virtual work-groups, however, it is not known at run-time with work-groups are concurrently running and hence the programming model cannot have dependencies across work-groups (figure 2.4).

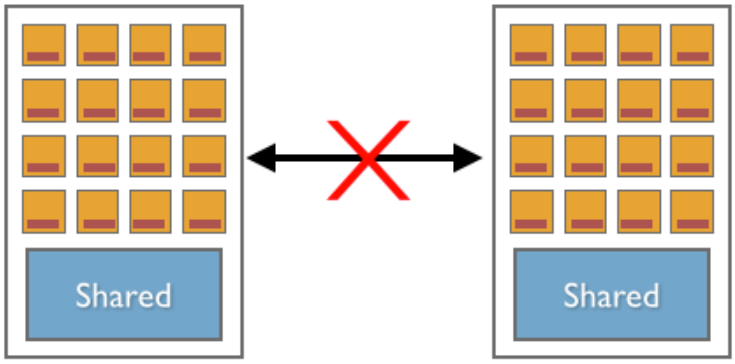


Figure 2.4 : Work-groups are not able to communicate with each other due to their arbitrary launch scheduling.

Recalling that work-groups are made up of large vectors of FPUs, serialization occurs when branching occurs in a warp/wavefront as seen in figure 2.5. Hence, best-practice programming with relation to work-groups and work-items is to prevent branching.

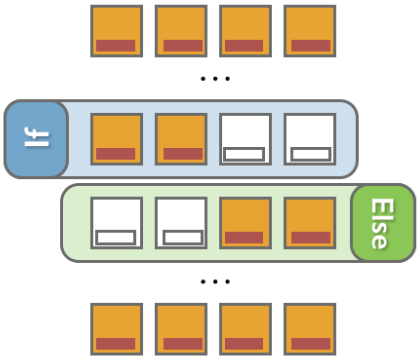


Figure 2.5 : If work-items in a warp/wavefront branch, the instructions are serialized and work-items alternate to compute their branching statement.

2.2.3.2 Memory

Going back to memory, this section goes over optimization practices for global and shared memory. For prior information on the memory types, the reader is asked to see section 2.2.1.

Starting with global memory, two optimizations can be followed: memory access and reuse of data. Looking at the GPU architecture in detail, global memory travels through the memory bus whose width is usually a multiple of 64 bits coinciding with the size of a float4, a primitive highly used in graphics computing. Recalling

that work-items are mapped to vector lanes in a large SIMD unit, coalesced reads, or consecutive work-items fetching consecutive data, makes use of the bus through vector loads as seen in figure 2.6. Without coalescing global memory fetches, data retrieval serializes on the bus, causing an increase in memory latency. However, the simplest way to prevent the high latency from global reads is to also reduce the number of reads. By making use of work-group arrangements, shared memory can be utilized by having all work-items read from global memory in parallel, thereby reducing global reads required by each individual work-item.

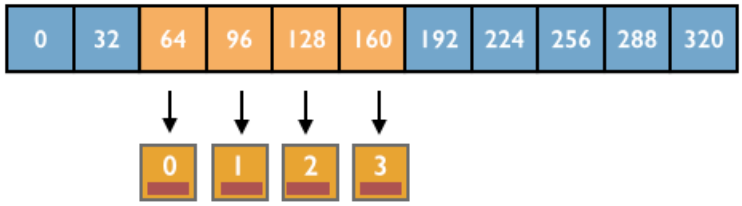


Figure 2.6 : Having consecutive work-items load from global memory consecutive data, otherwise known as coalescing reads, allows full use of the memory bus which vectorizes loads.

There are two main caveats with using shared memory which come from hardware limitations. The first limitation to shared memory is the amount allotted to a single streaming core. Similar to the L1 cache found in CPU processors, shared memory is a scarce resource. The second limitation relates with the distribution of memory and is also found in CPU cache. L1 cache is managed by memory banks which work in parallel but, individually, distribute data serially. A memory bank distributes address values with strides based on the number of banks as seen in figure 2.7.

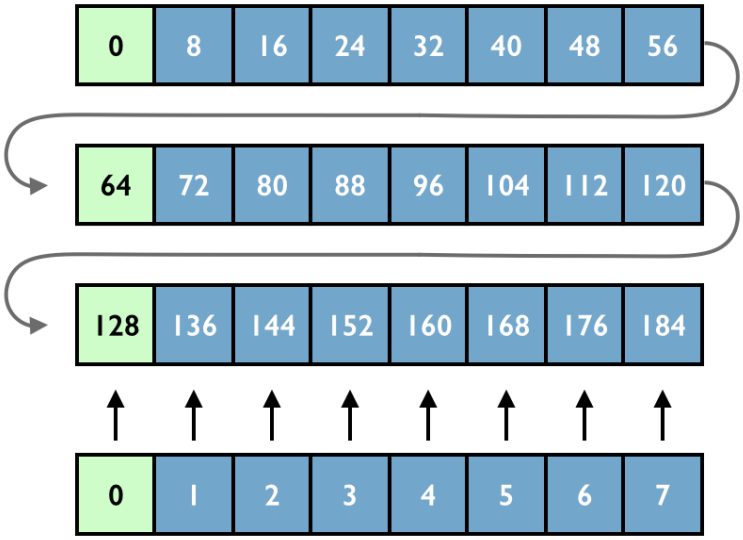
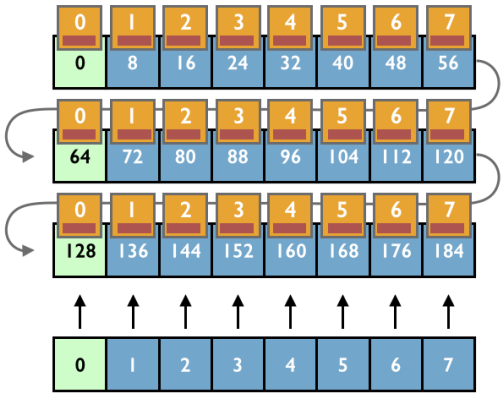


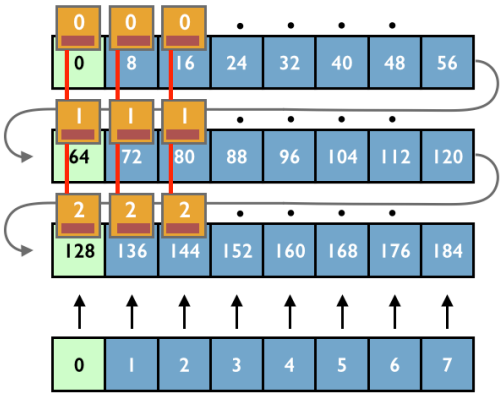
Figure 2.7 : Each memory bank distributes values from addresses with the same ending. Memory banks on GPUs number 32 or 64 in current architectures.

To make full use of the memory banks, a distribution of work-items on the memory banks is required as seen in figure 2.8a However, sharing memory banks with other work-items serializes data fetching, causing bank conflicts. For example, work-items reading the columns of the transpose of a matrix will require continuous loads as seen in figure 2.8b, causing memory bank conflicts.

A common optimization technique to avoid bank conflicts is to pad an array as seen in figure 2.9. By padding the array, stride accesses will have a cyclic pattern which attempts to avoid overlaps across work-items.



(a) Having work-items read shared memory in a coalesced manner parallelizes the use of the memory banks.



(b) Having work-items read shared memory in strides, such that each work-item reads consecutive data, forces each memory bank to serve all work-items and thereby serializing the memory banks.

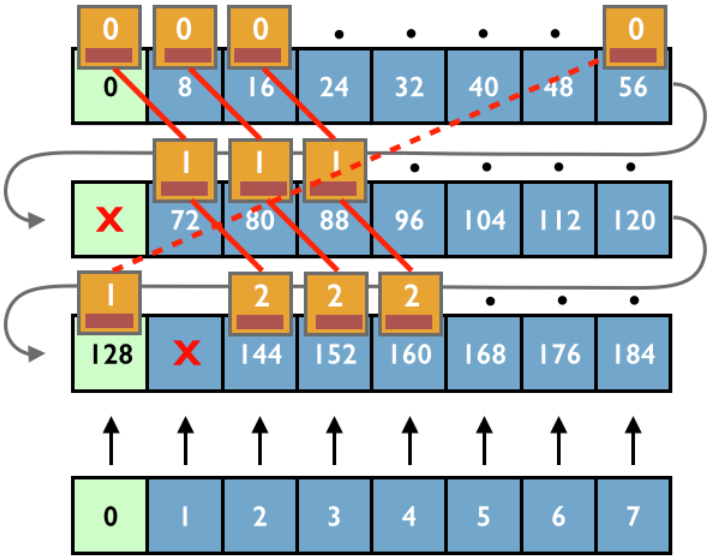


Figure 2.9 : The padding, shown by the ×, prevents the work-item overlap on a bank to prevent serializing data fetching.

2.3 Architecture Similarities

In summary, this chapter had an overview of best practice programming for both, CPU and GPU architectures. The common features in the CPU and GPU architectures overlap to give common optimizations, some which are listed in figure 2.10. This section will use the commonalities in CPU and GPU programming practices covered in this chapter to motivate the following chapter which goes over OCCA and its unified programming language.

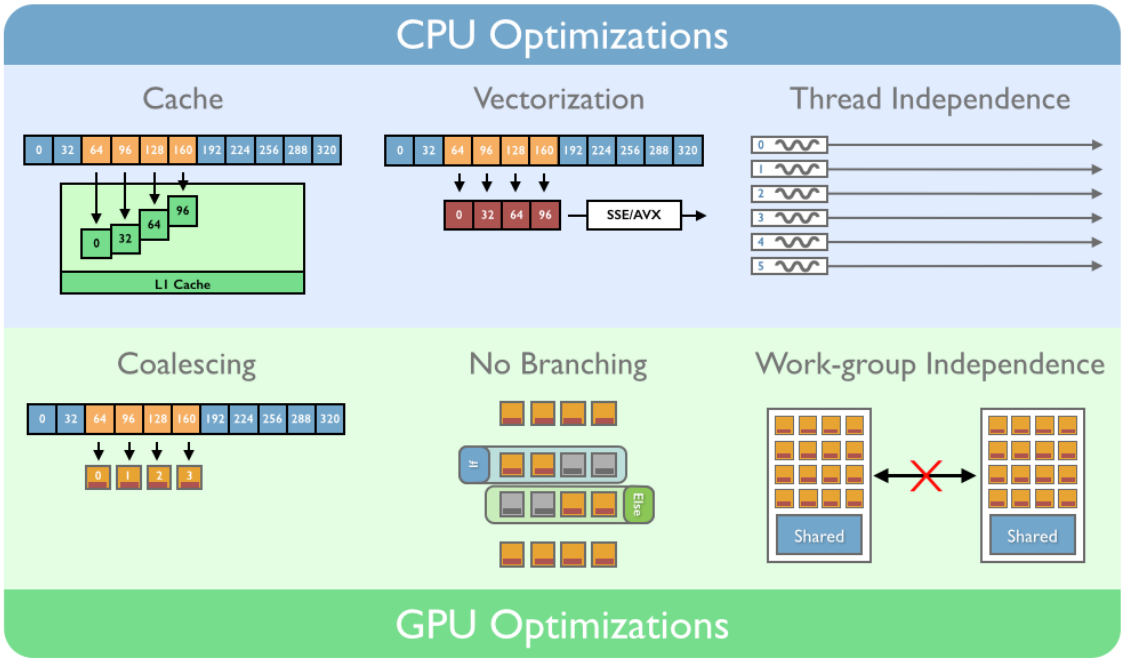


Figure 2.10 : The padding, shown by the red ×, prevents the work-item overlap on a bank to prevent serializing data fetching.

2.3.1 Memory Fetching

With memory fetching being a major bottleneck on both CPU and GPU architectures, similar approaches in programming for the architectures have been taken. In the CPU architecture, sequential fetching of data benefits from the multiple levels of cache available in modern processors. Although current GPU architectures have L1 and sometimes L2 cache, the user is able to manually manipulate L1 cache with the use of shared memory. Along with manual caching, the bus width on current GPUs are wide enough to support eight to twelve 32-bit addresses to be simultaneously fetched from memory.

The large-scale parallel hardware on GPUs tends to obtain large performance effects from minor optimizations as seen in [28]. When striving for high performance algorithms on GPU, memory fetching becomes a large obstruction. By managing memory loading on the GPU through coalescing reads, it can be expected that the same code on regular multi-core processors would benefit from cache hits.

2.3.2 Vectorization

Following memory fetching, computations that make use of data can improve from vectorization by launching the same instruction on large registers. Recalling that work-items are mapped to vector lanes on large SIMD units, GPUs natively vectorize instructions through each work-item computation. However, this vectorization is lost

once branching occurs, also a common trait in CPUs. While vectorization can still occur with branching, the programming practice to avoid branching on GPUs can also be taken advantage on regular processors to vectorize instructions.

2.3.3 Multi-threading

While perfect parallelism can only be observed in two individual hardware units without communication, a CPU processor achieves similar results with the use of multiple cores and pinned multi-threading. Similarly, GPUs achieve their large parallelism through the use of warp/wavefront groups with their own scheduler. Hence, programming multi-threaded code will benefit from code based on the work-group/work-item model seen in the GPU, where the native work-group parallelization on GPUs prevents communication across groups and hence across threads in multi-threaded code.

CHAPTER 3

OCCA

In this chapter, I discuss the main contribution of this thesis: the development of OCCA. Through the use of the optimization similarities seen in chapter 2, I worked on developing a macro-based language that currently supports OpenMP, OpenCL and CUDA.

The OCCA library consists of an API that abstracts API calls from supported languages and standards, together with a macro-based kernel language. The host API and kernel language found in OCCA were developed to maintain portability and performance together with platform-choice flexibility. First, a thorough description on the host front-end in OCCA will be given in section 3.1. Following is an explanation in section 3.2 on how the kernel formatting and language unwraps to match OpenMP, OpenCL and CUDA language specifications.

Following the OCCA explanation, a 2D finite difference (FD) is shown as a simple example that makes use of OCCA, together with pseudocode, OCCA code for the host and

kernel, together with performance results under different optimizations. Although the 2D finite difference code acts as proof-of-concept, I present mature numerical examples in chapter 4, chapter 5 and chapter 6 that use finite difference, spectral element method and discontinuous Galerkin method respectively. Throughout this chapter, I use OpenCL notation but include CUDA's complement inside double brackets such as

$$\text{OpenCL } [\![\text{CUDA}]\!]$$

to avoid focusing on one terminology. For further reference, an overview of OCCA keywords is provided in the appendix.

3.1 OCCA Host API

Aside from language-based libraries from OpenMP, OpenCL or CUDA, the OCCA host API is a stand-alone library. The independence allows OCCA to be combined with other libraries without conflict, as shown in figure 3.1 but maintains the flexibility to program a broad spectrum of architectures as seen in figure 3.2

Unlike many of the projects and codes already mentioned, the focus of OCCA is to provide tools for multi-platform computing, rather than code transformation. The following sections focus on the three key components that influenced the OCCA host API development: the platform device, device memory and device kernels.

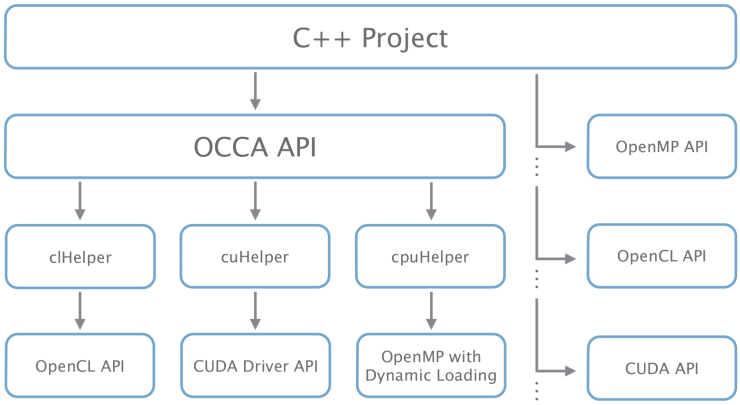


Figure 3.1 : OCCA wraps different language APIs and is non-conflicting with external libraries in either platform.

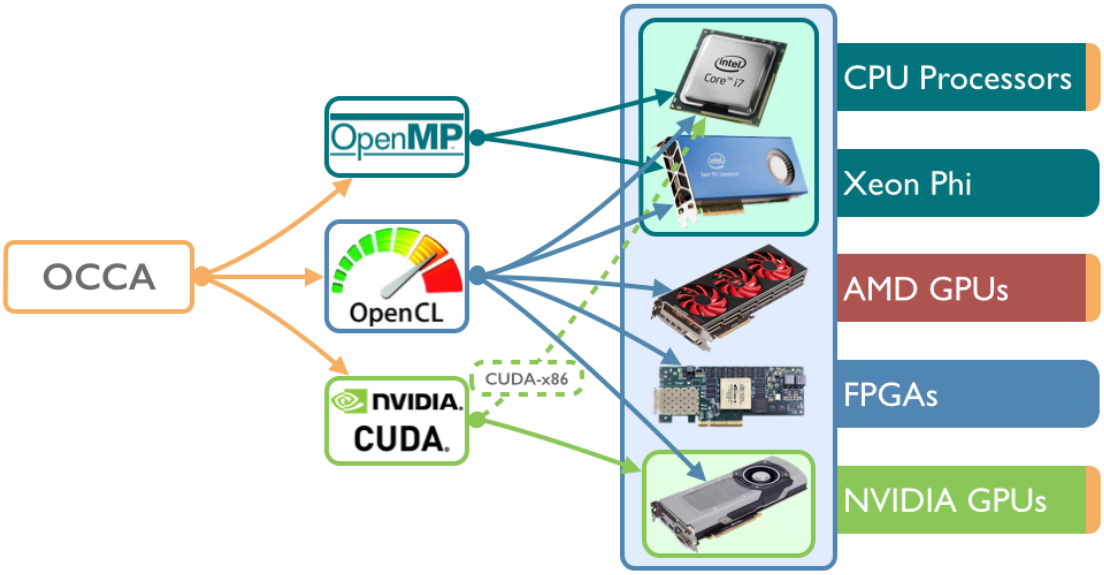


Figure 3.2 : With dynamic compilation, OCCA can be used to target different languages and therefore a spectrum of architectures.

3.1.1 OCCA device

An OCCA device acts as a layer of abstraction between the OCCA API and the API from supported languages. Due to the just-in-time code generation discussed in section 3.2, the platform target can be chosen at run-time. Enabled platforms are managed at compile-time in the case of unsupported platforms on the compiled architecture.

An OCCA device generates a self-contained context and command queue [[stream]] from a chosen device, being a socketed processor, GPU or other OpenCL supported devices such as a Xeon Phi or an FPGA. Asynchronous computations with multiple contexts can be achieved using multiple OCCA devices. The device's main purpose is to allocate memory and compile kernels for the chosen device.

3.1.2 OCCA memory

The OCCA memory class abstracts the different device memory handles and provides some useful information such as device array sizes. Although memory handling in OCCA facilitates host-device communication, the management of reading and writing between host and device, for performance reasons, is still left to the programmer. Having a class dedicated for device memory also allows the OCCA kernel class to differentiate and communicate between distinct memory types.

3.1.3 OCCA kernel

The OCCA kernel class unites device function handles with a single interface, whether for a function pointer (OpenMP), cl_kernel (OpenCL), or cuFunction (CUDA). When using the OpenCL and CUDA kernel handles, passing the arguments through their respective API is simple, but there are discrepancies when comparing to the OpenMP wrapper. For example, OpenCL and CUDA kernels work-items [[threads]] have access to work-group [[block]] and work-item counts implicitly. However, C++ functions only have access to the function scope and global namespace, requiring the work-group and work-item counts to be passed as macros or as an argument to the kernel.

3.2 OCCA Device API

The OCCA device API heavily relies on macros to give the user a clear front-end for programming device kernels. Defining grid and work-group size as a macro gives the kernel a fixed overall working size; changing the working size would require a kernel re-compilation. Hence, an extra argument is added to OCCA kernels, giving the flexibility to pass language-dependent data.

```
occaKernel void kernelExample(occaKernelInfoArg,
                              occaPointer datafloat* arg1
                              occaPointer datafloat* arg2){}
```

Listing 3.1: An example of an OCCA kernel header.

Macros were used to build the device API, letting the preprocessor translate the kernels to fit OpenMP, OpenCL, and CUDA standards. This approach was chosen, as opposed to the source-to-source solution used in other projects, to create a simple and lightweight library adding the flexibility to support other languages.

The `occaKernelInfoArg` in code listing 3.1 acts as a place-holder for external information needed in the kernel. For now, `occaKernelInfoArg` is only used in OpenMP mode to pass the computational grid and work-group sizes. The use of this place-holder will allow flexibility when enabling another platform or language. A comprehensive collection of all OCCA keywords and macro expansions can be found in the appendix.

3.2.1 Macro-based kernel generation

Using macros to connect the multiple languages creates a precise translation that is intuitive to the programmer without code-transformation subtlety. The OCCA keywords span keywords used in OpenMP, OpenCL and CUDA to cover major functionalities, such as implicit for-loops, barriers, etc.

Large projects tend to maintain backwards compatibility, while adding new features through updates or patches. Similarly, CUDA and OpenCL have kept a backwards-compatible model, a feature possible in OCCA with the addition of new OCCA keywords with the introduction of new macros.

3.2.2 Handling multi-threading architectures

The coding approach taken in both OpenCL and CUDA for coding multi-threading architectures is done at the granularity of a single work-item [[thread]]. Work-items are part of a work-group [[block]] that are able to communicate efficiently using shared memory.

```
for(int bZ = 0; bZ < get_num_groups(2); ++bZ){        // (1)
  for(int bY = 0; bY < get_num_groups(1); ++bY){
    for(int bX = 0; bX < get_num_groups(0); ++bX){
      // Shared memory is initialized here

      for(int tZ = 0; tZ < get_local_size(2); ++tZ){  // (2)
        for(int tY = 0; tY < get_local_size(1); ++tY){
          for(int tX = 0; tX < get_local_size(0); ++tX){
             // Scope of the OpenCL kernels
             // Register memory is initialized here
      }}}
}}}
```

Listing 3.2: The code listing expands the implicit for-loops found in OpenCL kernels. Loop grouping (1) expands multi-dimensional work-groups [[blocks]] and loop grouping (2) expands multi-dimensional work-items [[threads]].

```
for(int bZ = 0; bZ < gridDim.z; ++bZ){        // (1)
  for(int bY = 0; bY < gridDim.y; ++bY){
    for(int bX = 0; bX < gridDim.x; ++bX){
      // Shared memory is initialized here

      for(int tZ = 0; tZ < blockDim.z; ++tZ){  // (2)
        for(int tY = 0; tY < blockDim.y; ++tY){
          for(int tX = 0; tX < blockDim.x; ++tX){
             // Scope of the CUDA kernels
             // Register memory is initialized here
      }}}
}}}
```

Listing 3.3: Similar to code listing 3.2, the expansion of the implicit for-loops found in CUDA kernels is displayed. Loop grouping (1) expands multi-dimensional work-groups [[blocks]] and loop grouping (2) expands multi-dimensional work-items [[threads]].

The loop groupings `(1)` and `(2)` in code listing 3.2 and code listing 3.3 are implicitly invoked in OpenCL and CUDA by obtaining block information (`bX, bY, bZ`) and

thread information (`tX, tY, tZ`) through language-specific calls. Instances from the loop grouping `(1)` are generated independently (no order dependence is guaranteed in GPU programming) but each instance creates the threads seen in the grouping `(2)`. Because of this, loop formations were added to suit both, GPU programming and the explicit thread-based architecture of OpenMP. All OCCA keywords related to loop expansions can be found in table A.2 of the appendix,

```
occaOuterFor2{       // Loop grouping (1)
  occaOuterFor1{
    occaOuterFor0{
      // Shared memory defined here

      occaInnerFor2{ // Loop grouping (2)
        occaInnerFor1{
          occaInnerFor0{
      }}}
}}}
```

Listing 3.4: The OCCA programming model mirrors GPU programming, where group loopings (1) and (2) refer to work-groups [[blocks]] and work-items [[threads]] respectively.

where each `occaFor` macro is expanded with respect to the programming model chosen. In the case of GPU languages, the macros are defined as empty macros, encompassing whole kernel inside nested scopes that are optimized away. The kernel code expansion for the OpenMP mode is displayed in code listing 3.5.

```
//(1)
for(occaOuterId2 = 0; occaOuterId2 < occaOuterDim2; ++occaOuterId2){
  for(occaOuterId1 = 0; occaOuterId1 < occaOuterDim1; ++occaOuterId1){
    _Pragma("omp parallel for
            firstprivate(occaInnerId0, occaInnerId1, occaInnerId2,
                         occaDims0   , occaDims1   , occaDims2)")
    for(occaOuterId0 = 0; occaOuterId0 < occaOuterDim0; ++occaOuterId0){
      // Shared memory defined here

      // (2)
      for(occaInnerId2 = 0; occaInnerId2 < occaInnerDim2; ++occaInnerId2){
        for(occaInnerId1 = 0; occaInnerId1 < occaInnerDim1; ++occaInnerId1){
          for(occaInnerId0 = 0; occaInnerId0 < occaInnerDim0; ++occaInnerId0){
      }}}
}}}
```

Listing 3.5: Using a programming practice similar to GPU work-group [[block]] and work-item [[thread]] executions, our best choice to place the OpenMP pragma expansions were at the analogous inner-most work-group looping (1).

With the difference in work-item generation, having a work-item return in the middle of a kernel is problematic. Although it is bad practice to have single work-items `return` inside an OpenCL/CUDA kernel, `occaInnerReturn` is included in the OCCA keywords to allow GPU kernel work-items to `return` while having threads from OpenMP call `continue`. Barriers needed to sync work-items will split loop groups, as seen in code listing 3.6, which would require an OpenMP thread to `continue` in the successive loops if intended to call `return`.

3.2.3 Threaded barriers

Enabling the use of memory barriers is critical with memory latency becoming a more crucial issue due to new GPU architectures focusing on a greater stream processor to bandwidth ratio. For optimized codes, the use of faster memory types, such as local and register memory, is indicated. The thread-based granularity of GPU programming requires both, local and global barriers [[syncthreads]] as an important part of the GPU programming model. Many optimized kernels in GPUs make use of manual caching with local [[shared]] memory requiring barrier-support in OCCA [33].

For languages such as OpenMP that run work-groups in parallel but work-items in serial, intermediate barriers in the inner-loops have no synchronization like OpenCL

or CUDA that run work-items in parallel. Serial work-items require a split in inner-loops, as seen in code listing 3.6, to go through each iteration statement prior of the barrier. The OCCA barriers are given by `occaBarrier(occaLocalMemFence)` and `occaBarrier(occaGlobalMemFence)` for local and global memory respectively. Expansions to the keywords can be found in table A.6 of the appendix.

```
// Loop grouping (1)
// Shared memory defined here

occaInnerFor2{ // Loop grouping (2.1)
  occaInnerFor1{
    occaInnerFor0{
}}}

occaBarrier(occaLocalMemFence);

occaInnerFor2{ // Loop grouping (2.2)
  occaInnerFor1{
    occaInnerFor0{
}}}
```

Listing 3.6: Handling barriers in OCCA requires the loop splitting seen between (2.1) and (2.2) in this code listing. OpenMP handles outer-loops in parallel and inner-loops serially, needing a break only in inner-loops.

3.2.4 Registers and register arrays

As seen in the code listing 3.6, barriers cause a break in inner-loops. Having explicit loops when using OpenMP requires memory to be managed through discontinuous scopes. In other words, variable definitions in (2.1) are lost in scope and do not carry through to (2.2) in code listing 3.6, making register initialization non-trivial. Apart from scope visibility, overwriting becomes an issue with private variables defined outside of inner-loops. In OpenMP, each outer-loop is handled in parallel while inner-loops are executed serially, causing private register values to be overwritten. If the

variable `reg` in code listing 3.7 would be initialized as a regular variable, the value of `reg` would get overwritten by `occaGlobalId0` in each iteration.

To allow for proper register accessing, two OCCA keywords are introduced for work-item dependent registers and register arrays: `occaPrivate(type, name)` for and `occaPrivateArray(type, name, size)`. For GPU OCCA, the macros expand as a regular register or register array initialization. For OpenMP code, however, the macro expansion initializes an array with memory allocated for each work-item [[thread]] in a buffer. Operator overloading hides both, the work-item ID and required index in the array, treating `occaPrivate` and `occaPrivateArray` types as normal arrays. Similarly, device-dependent implementations can be done with the use of the OCCA keywords `occaCPU`, `occaGPU`, located in table A.8 of the appendix. Platform-dependent implementations can be managed with the use of `occaOpenMP`, `occaOpenCL`, and `occaCUDA` which are also found in table A.8 of the appendix.

```
occaKernel void kernelExample(occaKernelInfoArg, ...){
  occaOuterFor0{
    occaPrivate(int, reg);              // Register      : int reg;
    occaPrivateArray(int, regArray, 2); // Register Array: int regArray[2];

    // . . .

    occaInnerFor0{
      reg = occaGlobalId0; // reg would normally be overwritten by the loop
      regArray[0] = 0;     // regArray would also normally be overwritten
      regArray[1] = 1;
      // . . .
    }

    occaBarrier(occaLocalMemFence);

    occaInnerFor0{
      int i = reg;         // Allocating registers normally for only one scope
      int d = regArray[0];
      // . . .
    }
  }
}
```

Listing 3.7: The different memory types used in OCCA kernels can be seen in this code listing. `occaPrivate` and `occaPrivateArray` variables remove write-conflicts that might appear in the serial inner-loop execution when using OpenMP-mode.

3.3 Finite Difference Example

I present a simple OCCA example by describing a finite difference implementation with a brief PDE and discretization summary. Rather than optimization, the example's purpose is to show the construction of a simple OCCA project with both, host and device code. Performance studies on OCCA-supported languages, however, are the focus of the two projects, GNEK and RIDG, in chapter 5 and chapter 6.

Kernel performance for this example is compared between CPU architectures separately from GPU architectures with the GFLOP count (1e9 floating point operations) and bandwidth (memory transfer rate) as metrics. Results between the distinct CPU and GPU architectures on competing platforms (OpenMP vs OpenCL-CPU) and (OpenCL-GPU vs CUDA) are shown separately for each project. For the CPU architecture, OpenMP is compared with both, Intel's OpenCL platform and AMD's OpenCL platform on an Intel i7-3930K. For the GPU architecture, NVIDIA's OpenCL platform and CUDA are compared on NVIDIA's Titan together with AMD's OpenCL platform on the AMD Radeon 7990 card. Only one GPU processor is used on the dual-chip GPUs.

3.3.1 Governing Equations

The 2D time-dependent PDE governing the isotropy wave equation is given by

$$u_{tt} = u_{xx} + u_{yy}$$

where u is a scalar field representing acoustic pressure. Sound speed in the material is of constant value 1 for simplicity.

3.3.2 Discretization

The 2D domain is a standard square $[-1, 1] \times [-1, 1]$ and discretized into a structured grid of `w` node width and `h` node height. A first order second derivative stencil is used for stepping in time ($\{\frac{1}{\Delta t^2}, -\frac{2}{\Delta t^2}, \frac{1}{\Delta t^2}\}$). The spatial differential discretization is given by a stencil of size $2r+1$ in each dimension, where r is the stencil radius. The discrete stencil operator becomes

$$u_{n+1}(x_i, y_j) = (-2u_n + u_{n-1}) - \underbrace{\sum_{k=-r}^{r} \omega_k u_n(x_{i+k}, y_j)}_{u_{xx}} - \underbrace{\sum_{k=-r}^{r} \omega_k u_n(x_i, y_{j+k})}_{u_{yy}},$$

where u_n is the solution stored at $t = t_n$ using a $2r$ order FD stencil, containing $2r + 1$ nodes in each dimension. For simplicity in this example, no memory retrieval optimizations are made. To find an optimized GPU finite difference code for the wave equation, the reader is referred to [37].

Algorithm 1 Finite Difference Pseudocode.

1: **for all** (wg_i, wg_j) **do** { For each work-group in a 2D grid }
2: **for all** (i, j) **do** { For each thread in the 2D work-group}
3: $lap \leftarrow 0$
4: **for** $-r \leq k \leq r$ **do** { For all nodes in the 1D stencil }
5: $lap \leftarrow lap + \omega_k u_n(i+k, j) + \omega_k u_n(i, j+k)$ { Node contributions }
6: **end for**
7: $u_{n+1}(i, j) \leftarrow (-2u_n(i, j) + u_{n-1}(i, j) - \Delta t^2 * lap)$ { Store solution at $t = t_{n+1}$}
8: **end for**
9: **end for**

The discrete Laplacian in the finite difference method seen in algorithm 1 iterates over a 1D stencil for each dimension. The OCCA kernel code for the pseudocode in algorithm 1 is shown in code listing 3.8.

```
// External variables are defined as compiler directives
occaKernel void fd2d(occaKernelInfoArg,
                     occaPointer double *u1,
                     occaPointer double *u2,
                     occaPointer double *u3){
  occaOuterFor1{
    occaOuterFor0{
      occaInnerFor1{
        occaInnerFor0{
          const int i = occaGlobalId0;
          const int j = occaGlobalId1;

          const int id = j*w + i;      // w = nodes in the x direction
                                       // h = nodes in the y direction
          if( (i < w) && (j < h) ){    // Bounds check
            double lap = 0.0;

            const double r_u1 = u1[id]; // Global to register memory
            const double r_u2 = u2[id];

            for(int k = -r; k <= r; k++){
              const int nX = (i + k + w) % w; // Periodic Boundary
              const int nY = (j + k + h) % h; // Periodic Boundary

              lap += weight[r + k]*u1[j*w + nX]  // Iterate horizontal nodes
                   +  weight[r + k]*u1[nY*w + i]; // Iterate vertical nodes
            }

            u3[id] = (-2*r_u1 + r_u2 - dt*dt*lap);
          }
      }}
  }}
}
```

Listing 3.8: The OCCA finite difference kernel for the acoustic wave equation can be seen in this code listing. The pseudocode in algorithm 1 can be used for reference with this code listing.

The work-group/work-item programming style in OCCA requires a bounds check on the thread IDs as seen in code listing 3.8. In addition to the kernel code seen in code listing 3.8, code snippets are provided in code listing 3.9 to show the OCCA host API used in the FD example.

```
occa helper;
occaMemory o_u1, o_u2, o_u3;
occaKernel fd2d;

// . . .

void setupSolver(){
  // Setup occa helper
  // Model can be: "OpenCL", "CUDA", "OpenMP"
  helper.setup(model.c_str(), plat, dev);

  // Allocate device memory
  o_u1 = helper.createBuffer(u1); // u1, u2, u3 are 0 matrices of size w by h
  o_u2 = helper.createBuffer(u1);
  o_u3 = helper.createBuffer(u2);

  // Add kernel definitions
  string defs;

  helper.addDefine(defs, "r" , stencilRadius);
  helper.addDefine(defs, "w" , width);
  helper.addDefine(defs, "h" , height);
  helper.addDefine(defs, "dx", dx);
  helper.addDefine(defs, "dt", dt);

  // Build kernel
  size_t dims      = 2;
  size_t local[2]  = {16, 16};
  size_t global[2] = {local[0]*( (width  + local[0] - 1)/local[0] ),
                      local[1]*( (height + local[1] - 1)/local[1] )};

  fd2d = helper.buildKernel("fd2d.occa", "fd2d", defs);
  fd2d.setThreadArray(global, local, dims);
}

void timestep(){
  currentTime += dt;

  fd2d(o_u1, o_u2, o_u3, currentTime);

  // Shift timestep solutions to: u1 <- u2 <- u3
```

```
  o_u1.swap(o_u2); // Swaps memory handle between u1 and u2
  o_u2.swap(o_u3); // Swaps memory handle between u2 and u3
}

void main(){
  loadVariables(); // Load all variables (model, width, height, ...)
  setupSolver();

  while(true)
    timestep();
}
```

Listing 3.9: The OCCA host code used for the finite difference code is in this listing.

3.3.3 Finite Difference Performance

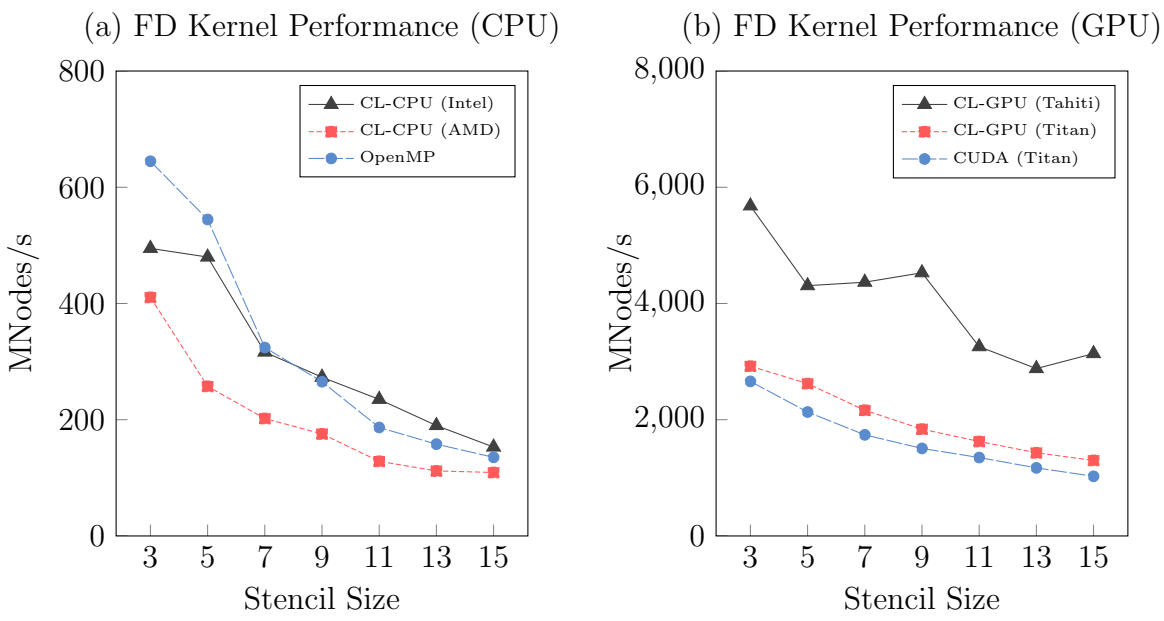


Figure 3.3 : Performance in MNodes/s of the FD kernel in code listing 3.8 are shown in (a) and (b). Results shown in figure (a) were obtained on three CPU platforms on an Intel i7-3930K 6-core processor. Results shown in figure (b) were obtained on an NVIDIA Titan card and AMD Radeon 7970.

The performance of finite difference implementations is usually given by the millions of nodes processed per second. Taking the standard performance metric in finite difference implementations, figure 3.3 uses MNodes/s to gauge performance. Figure (a) shows similar performance between OpenMP and Intel's OpenCL platform, while AMD's OpenCL platform does not seem to vectorize well. As for GPU comparisons, figure (b) results show OpenCL performs better than CUDA in NVIDIA's Titan card while OpenCL on AMD's Tahiti outperforms the Titan results for this case.

CHAPTER 4

Vertical Transverse Isotropy - Finite Difference

Migration schemes are of great importance in seismic exploration for detecting subsurface reflections. Initial seismic imaging methods used integral methods, such as Kirchhoff migration, and are still known for their lightweight computing requirements [47]. With computational performance growths, it became feasible to use wave equation migration (WEM) methods; methods that are more computational intensive than integral methods, but are able to capture greater dip angles in seismic images [5]. Reverse time migration (RTM) doubles the wave simulations compared with standard WEM schemes by simulating wave migrations from sources and receivers, but acquires scattering from seismic refractions in the velocity model.

With the popularity of finite difference methods for wave migration methods, it is of interest to have a finite difference example utilizing OCCA. In this chapter, I discuss the OCCA version of a highly tuned CUDA kernel commonly found in seismic imaging kernels and the optimization techniques exposed in the process of translating

the kernel to OCCA. Following the optimization techniques, I show performance results for the alternative optimization methods with the native CUDA kernel and the translated OCCA kernel. Performance results are compared between the CUDA optimized code on a GTX Titan NVIDIA GPU and the supported platforms offered in OCCA, including the same GTX Titan GPU, an AMD Radeon 7970 GPU and an Intel 6-core i7-3930k processor.

4.1 Wave Equation Discretizations

The wave equation on an isotropic media is governed by the equation,

$$p_{tt} = c^2 \Delta p$$

where p is the pressure wave-field and c is the wave velocity. In seismic exploration, however, sedimentary layers make up the domain of interest whose material properties vary with direction. Two different anisotropy models were introduced, vertical transverse isotropy (VTI) and tilted transverse isotropy (TTI).

Adding anisotropy, the VTI models materials with symmetry along the z axis (or depth axis). The VTI model of Duveneck et al [15] is considered, giving the system of equations

$$\begin{aligned} p_{tt} &= c^2\Big((1+2\epsilon)\,(p_{xx}+p_{yy}) + \sqrt{1+2\delta}(q_{zz})\Big) \\ q_{tt} &= c^2\Big(\sqrt{1+2\delta}\,(p_{xx}+p_{yy}) + q_{zz}\Big), \end{aligned}$$

where p and q are the horizontal and vertical stress components, c is the acoustic velocity of the material and ϵ and δ are the Thomsen anisotropy parameters [44].

Applying a first order explicit time-stepping scheme, the discretized PDE takes the form

$$\begin{aligned} p^{n+1} &= 2p^n - p^{n-1} - dt^2c^2\Big((1+2\epsilon)\left(p^n_{xx} + p^n_{yy}\right) + \sqrt{1+2\delta}(q^n_{zz})\Big) \\ q^{n+1} &= 2q^n - q^{n-1} - dt^2c^2\Big(\sqrt{1+2\delta}\left(p^n_{xx} + p^n_{yy}\right) + q^n_{zz}\Big). \end{aligned}$$

Applying a stencil of radius r for the spatial derivative discretization, the finite difference method becomes

$$\begin{aligned} p^{n+1}(x_i, y_j, z_k) &= 2p^n(x_i, y_j, z_k) - p^{n-1}(x_i, y_j, z_k) \\ &\quad -dt^2c^2 \sum_{s=-r}^{r} (1+2\epsilon)\left(w^k_x p^n(x_{i+s}, y_i, z_k) + w^k_y p^n(x_i, y_{j+s}, z_k)\right) \\ &\quad -dt^2c^2 \sum_{s=-r}^{r} \sqrt{1+2\delta}(w^k_z q^n(x_i, y_j, z_{k+s})) \\ q^{n+1}(x_i, y_j, z_k) &= 2q^n(x_i, y_j, z_k) - q^{n-1}(x_i, y_j, z_k) \\ &\quad -dt^2c^2 \sum_{s=-r}^{r} \sqrt{1+2\delta}\left(w^k_x p^n(x_{i+s}, y_i, z_k) + w^k_y p^n(x_i, y_{j+s}, z_k)\right) \\ &\quad -dt^2c^2 \sum_{s=-r}^{r} w^k_z q^n(x_i, y_j, z_{k+s}), \end{aligned}$$

where w^k_x, w^k_y, and w^k_z are the finite difference stencil weights in time-step n.

4.2 Reverse Engineering CUDA Optimizations

The conversion from the CUDA-optimized source code for the forward wave propagation to OCCA did not present much difficulty. However, in the process of achieving relative performance on different platforms, I noted features in CUDA that would not adapt to the OCCA programming model. This section goes over the translation process from CUDA to OCCA and explains the best-practice programming concepts for programming in OCCA. In addition, because the parallel algorithm proposed by [37] is highly optimized to make use of the different layers in the GPU memory hierarchy, I'll discuss the memory layout of the problem and follow by noting which memory layer is best suited for each data load.

Using the fact that the usual domain for most seismic imaging problems is in the shape of a rectangular prism, the discretization of the domain becomes a grid of size $N_x \times N_y \times N_z$. The VTI finite difference kernel pseudocode is given by

```
Partition the top plane of the grid into B_x × B_y blocks of size w × h
For time-step n = 0, 1, ..., time-Steps
    For each block (b_i, b_j)                                    (1)
        For n = 0, 1, ..., N_z
            For each point (i, j, k) such that
                (b_i ≤ i < b_i + w) and (b_j ≤ j < b_j + h)       (2)
                Update p^{n+1}(i, j, k) and q^{n+1}(i, j, k)
            End For // Point Update
        End For     // Traversing depth
    End For         // Iterating over blocks
```

```
End For              // Computing a time-step update
```

Listing 4.1: For each time-step, the 2D blocks at the top of the structured grid sweep in the z direction and update all points in the current z plane.

Using the GPU programming model of work-groups and work-items, computational work needs to be divided into independent work-groups containing work-items that execute the same instruction (i.e. no branching). Hence, step (1) is computed at the work-group level with each of the group's work-item computes the updates in step (2).

The computations required in step (1) in code listing 4.1 are dependent on the order of accuracy of the finite difference scheme. Using a $2r$ order finite difference method in space requires a stencil with radius r in each dimension. Updating a single point in a $2r$ order scheme requires data from $2r$ neighboring points in each of the x, y, and z directions as shown in figure 4.1. The fine granularity computations for updating a single point in the finite difference method suit the work-item level of programming.

There are three basic arrangements of points for a work-group, shown in figure 4.2; either as a one dimensional line, a two dimensional square or a three dimensional block. Given the arrangement of points, neighboring dependencies from the finite difference stencil produce different halo regions as shown in figure 4.3, revealing the external data required for updating all points inside the group.

Figure 4.1 : Finite difference stencil.

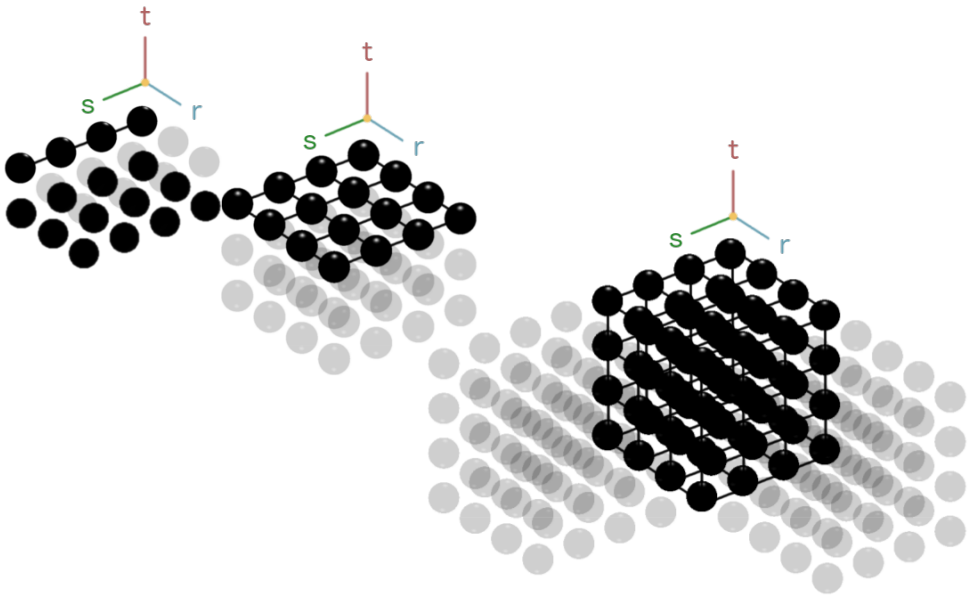


Figure 4.2 : Work-groups arrangements as a 1D line, a 2D square or a and 3D cube.

Using the work-item groupings, we're able to exploit the data redundancy by sharing data loads. In order to share data between work-items, the use of shared memory is required. Benefits of using shared memory include parallel data loads from global memory and a decrease in memory-fetching latency. However, because the amount

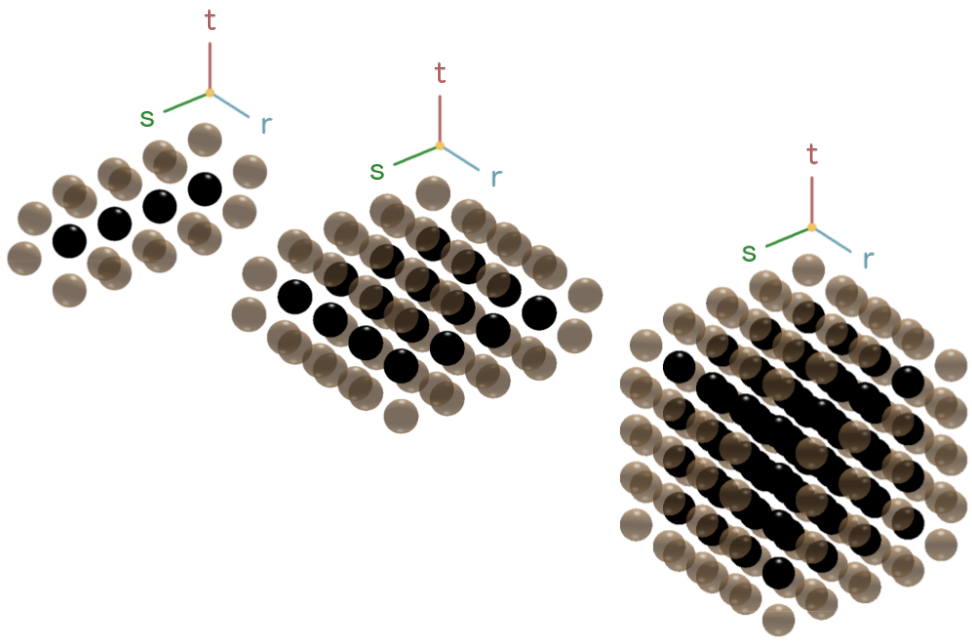


Figure 4.3 : Halo region shown for the different work-group arrangements: as a 1D line, a 2D square or a and 3D cube.

of shared memory per compute unit in the GPU is limited, the number of work-items is largely dictated by the amount of shared memory needed. Hence, for each arrangement of points, I examine the ratio between the required shared memory and the number of work-items in the group.

For a one dimensional grouping of N points, the number of unique points required for updating the group is $(4r + 1)N + 2r$. Sharing the data in the group and halo region, an approximate load of data from $4r + 1$ points by each work-item.

Given a two dimensional grouping of $N \times N$ points requires data from $(2r + 1)N^2 + (4r)N$ points to update each group. There is an approximate load of data from $2r + 1$ points by each work-item if we truncate the low power term $(4r)N$.

Lastly, using a three dimensional grouping of $N \times N \times N$ points requires the data for $N^3 + (6r)N^2$ points to update each group. The approximate point load average is now independent of r.

Although shared memory is limited, data fetching takes precedence due to the high reuse of shared memory. Hence, looking at the complex shape of the halo region, it is more efficient to load data as a structured grid. The structured format benefits from two optimization techniques often used in GPU programming: removing branching statements for accessing halo data and determining memory bank usage ahead of time. Unfortunately, the shared memory requirements become $(2r+1)^2(N+2r)$, $(2r+1)(N+2r)^2$ and $(N+2r)^3$ for 1D, 2D and 3D groupings respectively.

The workaround on the limits of shared memory in [37] was to use a register array to store the needed data in the z dimension of data (slowest dimension) for each point, sacrificing parallel data-loading. Was the register array to be placed in shared memory, work-items dedicated to neighboring points in the same dimension would be able to reuse the data. Rather than having other local work-items reuse the loaded data, each work-item can work on multiple points in the z direction by rolling the register array as seen by [37], amortizing data load to $O(1)$ loads.

Therefore, 2D groupings are used and the working domain consists of a prism of points elongated in the z direction as seen by [37].

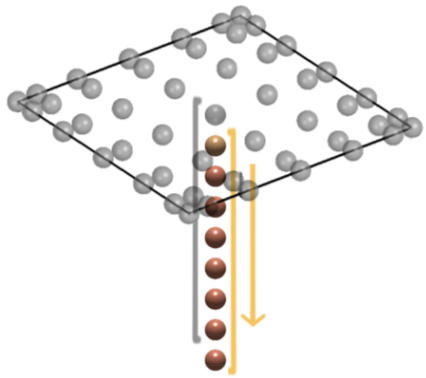


Figure 4.4 : Register rolling re-uses loads from global memory by only requiring one global load per sweep and re-using the other $N - 1$ pre-loaded values.

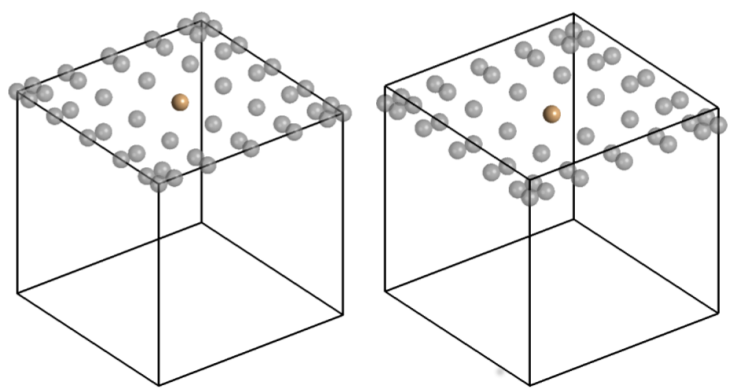


Figure 4.5 : In each sweep, the work-group iterates one point in the sweeping direction.

The group of work-items iterate on the z dimension, loading the plane data and it's halo region in parallel at every iteration and using the register array to compute the derivative in the z direction.

Due to the plane-moving in the algorithm, the work-item shifts in the z direction, keeping the x and y damping coefficients constant for a given work-item; however, the z damping coefficients change for every update. Making use of the structured grid domain discretization, the tensor products of the damping weights in the x, y and z

directions create the proper damping for a given point (i.e. point p_{ijk} is damped by the scalar produced by `xDamp[i]*yDamp[j]*zDamp[k]`).

The notation used in OCCA code and pseudocode uses the following prefix notation to specify the memory location for each variable:

Type of variable	Prefix
Private variable	`p_`
Private array variable	`pa_`
Shared variable	`s_`
Global variable	`g_`
Constant variable	`c_`

The pseudocode of the resulting algorithm for a given work-group is given by

```
//---[ Defines ]------------------------
//   Global work-item id: (gi,gj)
//   Local work-item id : (li,lj)
//   Work-group size    : (Bx,By)
//
//   p^{n+1}, p^n, and p^{n-1} are denoted by P2, P1 and P0
//   q^{n+1}, q^n, and q^{n-1} are denoted by Q2, Q1 and Q0
//
//   wxy and wz are stencil weights for x-y and z
//   dampX, dampY, and dampZ hold PML damping coefficients
//=====================================

p_dampXY = c_dampX[gj][gi] * c_dampY[gj][gi];

For z = 0, 1, ..., N_z

   // Store global data in shared memory
   s_P1[li][lj] = P1[gi - r][gj - r];

   if(li < 2*r){
      s_P1[lj][li + Bx] = P1[z][gj - r][gi + Bx - r];
      s_P1[lj + By][li] = P1[z][gj + By - r][gi - r];

      if(lj < 2*r)
         s_P1[lj + By][li + Bx] = P1[z][gj + By - r][gi + Bx - r];
   }
```

```
    // Register rolling
    for(int i = 0; i < 2*r; ++i)
       pa_Q1[i] = pa_Q1[i + 1];

    pa_Q1[2*r] = g_Q1[z][gj][gi];

    p_Qtt = 2*pa_Q1[r] - g_Q0[z][gj][gi];
    p_Qz  = c_wz[r]*pa_Q1[r];

    for(int i = 0; i < r; ++i){
       p_Qz += c_wz[i]*pa_Q1[i]
            +  c_wz[2*r - i]*pa_Q1[2*r - i];

    p_dampXYZ = p_dampXY * c_dampZ[z][gj][gi];

    localBarrier(); // Wait for shared memory loads

    p_Ptt = 2*s_P1[lj + r][li + r] - g_P0[z][gj][gi];
    p_Pxy = 2*c_wxy[r]*s_P1[lj + r][li + r];

    for(int i = 1; i <= r; ++i)
       p_Pxy += c_wxy[i]*(s_P1[lj + r - i][li + r]
                          + s_P1[lj + r + i][li + r]
                          + s_P1[lj + r][li + r - i]
                          + s_P1[lj + r][li + r + i]);

    p_vx = g_vx[z][gj][gi];
    p_vy = g_vy[z][gj][gi];
    p_vz = g_vz[z][gj][gi];

    g_P2[z][gj][gi] = (p_Ptt + p_vx*p_Pxy + p_vz*p_Qz) * p_dampXY;
    g_Q2[z][gj][gi] = (p_Qtt + p_vn*p_Pxy + p_vz*p_Qz) * p_dampXY;

    localBarrier(); // All threads are done with shared memory

 End For     // Traversing depth
```

4.3 OCCA Kernel Tuning

In this section, I discuss the steps taken to produce an equally performing kernel using OCCA, despite lacking some features found in the CUDA language. Making use of the similarities between CUDA and OCCA, the first step is to create a working CUDA kernel without the CUDA-specific features, followed by the OCCA convection.

4.3.1 Hinting Constant Pipeline

Apart from using shared and register memory, [37] made use of constant memory which uses a different pipeline to stream data to the processing units. Specifically, constant memory acts as a useful buffer when all threads access the same entry at a given time. The material and damping coefficients used to describe the velocity model and damp outgoing waves outside the boundary are, in this problem, small enough to fit the constant buffer.

All constant memory buffers must have compiler-determined size and be defined in the global namespace as seen in code listing 4.2. Rather than using `cudaMemcpy`, the programmer must use `cudaMemcpyToSymbol`, or call `cuModuleGetGlobal` using the driver API, to work with constant memory.

```
__constant__ c_buffer[100];

__global__ void kernel(...){ ... }
```

Listing 4.2: An example of constant memory in CUDA.

Using a pre-defined buffer in the global namespace would work with the OpenMP-mode. Through the use of `dlopen`, the address of a global memory buffer can be obtained. However, OpenCL does not support accessing addresses from arrays allocated in the global namespace, limiting data to be passed as arguments to kernels.

There are two approaches to hinting the compiler to use the constant pipeline. The first approach is to decorate the kernel arguments with `const` and `restrict`, hinting

the compiler to use the constant pipeline [38]. The second approach is to use of CUDA's `__ldg` intrinsic, with it's use seen in [38]; however, it is only available on GPUs with compute capability 3.5 and greater.

```
__global__ void kernel(const float * __restrict__ ptr){
    const float r_ptr = __ldg(&ptr[0]);
}
```

4.3.2 Performance Comparisons

Using intrinsics to load data becomes language specific and conflicting with the simplicity OCCA tries to offer. To inspect the performance contribution from constant memory, I display the performance comparisons in table 4.3 between: the original CUDA code which makes use of constant memory; replacing constant memory with global memory with and without parameter decorations; and lastly, using shared memory to replace constant memory.

Method	OCCA:CUDA	Native	Native (No Constant)	Native (Decorations)
NVIDIA K40	2468	2598	1971	1756
NVIDIA Titan	2773	2909	2398	2182

Table 4.1 : Performance comparisons (MNodes/s) of the CUDA VTI finite difference kernel using different memory-handling methods.

Replacing constant memory with global memory came with a loss of ($\approx 25\%$) performance and the parameter decorations reduced that to ($\approx 15\%$). However, the use

of shared memory rivaled the use of constant memory with a ($\approx 0\%$) loss and hence the memory-handling model used in the construction of the OCCA kernel. Using the highest performing case (512 x 512 x 256), figure 4.3 displays the results between CUDA and OCCA using the supported models and different architectures.

Device\Model	OCCA		
	OpenCL		OpenMP
	AMD	Intel	
Intel i7-3930k	64	131	90
Intel Xeon E5520	49	108	67

Table 4.2 : Performance comparisons (MNodes/s) of the VTI finite difference kernel using different combinations of languages and GPU devices.

Device\Model	OCCA			Native
	OpenCL		CUDA	CUDA
	AMD	NVIDIA		
NVIDIA K40	×	3013	2468	2598
NVIDIA Titan	×	3383	2773	2909
AMD Radeon 7970	3429	×	×	×

Table 4.3 : Performance comparisons (MNodes/s) of the VTI finite difference kernel using different combinations of languages and GPU devices.

CHAPTER 5

gNek

In this chapter, I focus on gNek which uses a spectral element method (SEM) algorithm that, utilizing occa, numerically solves for the Navier-Stokes equations in heterogeneous platforms. gNek is based on Nek5000 [17], an open source project led by Paul Fischer in Argonne National Lab, and tailors the algorithms for GPU acceleration ([19], [18]). I first discuss the PDE and splitting method which separated linear and non-linear terms. Afterwards, I discuss the discretization and formulation of the operators in each splitting step of the Navier-Stokes equations. Following is the parallel implementation of the spectral element method which discusses the coupling and decoupling of the mass matrix and stiffness operators. Lastly, I mention test cases that ran on gNek showing local convergence, scaling and performance on different architectures.

5.1 Introduction

As one of the fundamental equations in computational fluid dynamics (CFD), Navier-Stokes (5.1) derived PDEs have been heavily studied and are of interest due to modeling fluids with different viscosity, such as fluid and gas. The equations are given by

$$u_t + (u \cdot \nabla)u = -\nabla p + \nu \Delta u, \tag{5.1}$$

where u corresponds to the unknown velocity field, p to the scalar pressure field and ν is a given kinematic-viscosity parameter. The viscosity scalar is usually associated with the inverse of the Reynolds number, a parameter non-dimenionalizing the equations and identifying fluid properties such as viscosity. For this example, we add an incompressibility constraint given by $\nabla \cdot u = 0$. Using the splitting scheme by [32], the nonlinearity term $(u \cdot \nabla)u$ is treated explicitly and reduces the remaining terms into two series of elliptic solves.

$$\begin{aligned} \tfrac{1}{\Delta t}\left(\tilde{u} - u^n\right) &= -(u^n \cdot \nabla)u^n \\ \tfrac{1}{\Delta t}\left(\tilde{\tilde{u}} - \tilde{u}\right) &= -\nabla p^{n+1} \\ \tfrac{1}{\Delta t}\left(u^{n+1} - \tilde{\tilde{u}}\right) &= v \Delta u^{n+1} \end{aligned} \tag{5.2}$$

The decoupled equations in now introduce two intermediate velocities $(\tilde{u}, \tilde{\tilde{u}})$. An advantage to this decoupled form comes from not requiring a non-linear solver to treat $-(u^n \cdot \nabla)u^n$ implicitly, yet allows for a high (?) order implicit solver for the $-\nabla p$ and $v\Delta u$ right-hand sides (RHS). Several publications, such as [26], indicate the treatment of the nonlinear term does not affect stability and therefore can be treated explicitly for simplicity.

The split form of the PDE results in three equations for the physical phenomena of advecting fluid velocities, adding force from pressure stress, and diffusion of fluid velocities. The advection step

5.2 Discretization

For a physical simulation, we start with a domain of interest, represented by Ω, and it's boundary, represented by $\Gamma = \partial\Omega$. Because we seek a numerical solution to approximate physical phenomena, a discretized approximation of the domain is used, $\Omega_h \approx \Omega$; similarly, the discretized boundary is given by $\Gamma_h \approx \Gamma$. For this spectral element method, the discretized domain is partitioned into non-overlapping conforming hexahedral elements

$$\Omega_h = \bigcup_{e=1}^{E} D^e.$$

The hexahedral reference element is denoted by $\widehat{D} = [-1, 1]^3$, allowing for a simplified

tensor product structure at each element. The elements are straight-sided without tangling, allowing for an affine mapping between a given element D^e and the reference element D.

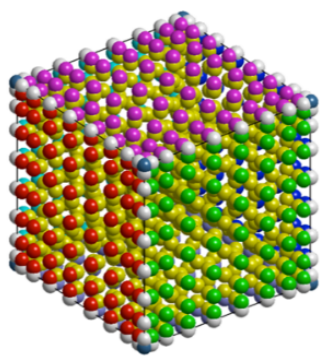


Figure 5.1 : Example hexahedral spectral element using a 9th order discretization.

5.2.1 Variational Form

Aside from the explicit non-linear advection step, two of the three steps used in the splitting scheme generalize to the screened Coulomb equation, being

$$-\Delta u + \lambda^2 u = f.$$

The choice of λ and f in the pressure step is

$$\begin{aligned} \lambda^2 &= 0 \\ f &= \tfrac{1}{\Delta t}\tilde{u}, \end{aligned}$$

whereas in the diffusion step, λ and f take the form of

$$\begin{aligned} \lambda^2 &= \tfrac{1}{v\Delta t} \\ f &= \tfrac{1}{v\Delta t}\tilde{\tilde{u}}. \end{aligned}$$

Although $\lambda = 0$ for the pressure step, the Laplacian operator preserves the positive-semidefinite property in the system. If we take some test space V, the variational form of (5.2.1) can quickly be derived as follows

$$\begin{aligned} -\left(\Delta u, \phi\right)_\Omega + \lambda^2 \left(u, \phi\right)_\Omega &= \left(f, \phi\right)_\Omega \quad \forall \phi \in V \\ \left(\nabla u, \nabla \phi\right)_\Omega + \lambda^2 \left(u, \phi\right)_\Omega - \left(\nabla u \cdot n, \phi\right)_{\partial\Omega} &= \left(f, \phi\right)_\Omega \quad \forall v \in V, \end{aligned}$$

reducing the null space to constant modes which are projected out.

5.2.2 Test and Trial Spaces

The test and trial function spaces are both notated by V, a piecewise polynomial space of degree N that is C^0-continuous across elements. Due to the continuity enforced by the spectral element method, a tensor product of the one dimensional Gauss-Lobatto-Legendre (GLL) nodes is used for the nodal-basis in V.

With the two nodes at each end-point, the tensor product of GLL nodes allows continuity to be enforced across neighboring elements without further interpolation. Define the GLL nodes as $r_i \in [-1, 1]$ for $i = 1, 2, \ldots, N+1$, then the associated nodal

basis is given by the Lagrange functions

$$\phi_i(r) = \prod_{\substack{j=1 \\ j!=i}}^{N+1} \left(\frac{r - r_j}{r_i - r_j} \right).$$

This choice of Lagrange functions gives the Kronecker delta property

$$\phi_i(r_j) = \delta_{ij}.$$

Extending this basis to $\mathbb{R}^2$ and $\mathbb{R}^3$ gives

$$\begin{aligned} \phi_{ij}(r,s) &= \phi_i(r)\phi_j(s) \\ \phi_{ijk}(r,s,t) &= \phi_i(r)\phi_j(s)\phi_k(t) \end{aligned}$$

Note the Kronecker delta relations still hold in multiple dimensions, more specifically

$$\begin{aligned} \phi_{ij}(r_a, s_b) &= \phi_i(r_a)\phi_j(s_b) \\ &= \delta_{ia}\delta_{jb} \\ \phi_{ijk}(r_a, s_b, t_c) &= \phi_i(r_a)\phi_j(s_b)\phi_k(t_c) \\ &= \delta_{ia}\delta_{jb}\delta_{kc}. \end{aligned}$$

The partial derivatives in the Navier-Stokes equations come from gradients and divergence, requiring the calculations of u_x, u_y, u_z. Using chain-rule, the partial derivatives

can be represented by the reference dimensions r, s, t as

$$\begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix} \begin{bmatrix} u_r \\ u_s \\ u_t \end{bmatrix}$$

Unfortunately, the Kronecker delta simplification does not hold for the Lagrange basis derivatives $\left(\phi_i'(r_j) \neq \delta_{ij}\right)$. There is another simplification in the partial derivatives on multiple dimensions, however, due to the tensor product of these basis. Because my spectral element implementation is only in $\mathbb{R}^3$, I'll only focus on the three dimensional expansions; however, simplifications follow in $\mathbb{R}^2$ as well.

$$\begin{aligned} \partial_r \phi_{ijk}(r,s,t) &= \phi_i'(r)\phi_j(s)\phi_k(t) \\ \partial_s \phi_{ijk}(r,s,t) &= \phi_i(r)\phi_j'(s)\phi_k(t) \\ \partial_t \phi_{ijk}(r,s,t) &= \phi_i(r)\phi_j(s)\phi_k'(t) \end{aligned} \tag{5.3}$$

Using the simplification in equation (5.3), the partial derivatives simplify to

$$\begin{aligned} \partial_r \phi_{ijk}(r_a, s_b, t_c) &= \phi'(r_a)\delta_{jb}\delta_{kc} \\ \partial_s \phi_{ijk}(r_a, s_b, t_c) &= \phi'(s_b)\delta_{ia}\delta_{kc} \\ \partial_t \phi_{ijk}(r_a, s_b, t_c) &= \phi'(t_c)\delta_{ia}\delta_{jb} \end{aligned}$$

The simplifications that arise from choosing GLL nodes to derive the basis on V,

combined with the choice of GLL-based quadrature allows for further simplification in the implementation of our numerical scheme.

5.2.3 Discretized Operators

One of the advantages of using GLL interpolation nodes is the resulting lumped mass matrix. Let w_i be the corresponding quadrature weight for the nodal basis on the GLL node r_i. The local mass matrix becomes

$$\begin{aligned} M_{ij}^e &= \left(\phi_i^e, \phi_j^e\right) \\ &\approx \sum_k w_k \phi_i^e(x_k)\phi_j^e(x_k) \\ &= w_k \delta_{ij}, \end{aligned} \tag{5.4}$$

where the approximation comes from aliasing of integrating a polynomial of degree $2N$ with quadrature accuracy of order $2N-1$. The quadrature weights, w_i, are non-zero leading to an easily invertible diagonal matrix.

Although the stiffness matrix is the discretized operator is

$$(\nabla u, \nabla \phi) + \underbrace{\lambda^2 (u, \phi)}_{\text{Mass Matrix}},$$

I'll only discuss the diffusion operator, $(\nabla u, \nabla \phi)$, since the second term is a scaling of M. From equation (5.4), I can represent ∇u in the reference space, reducing $\nabla u^T \nabla \phi$

to

$$
\begin{aligned}
\nabla u^T \nabla \phi &= \begin{bmatrix} u_x, & u_y, & u_z \end{bmatrix} \begin{bmatrix} \phi_x \\ \phi_y \\ \phi_z \end{bmatrix} \\
&= \left(\begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix} \begin{bmatrix} u_r \\ u_s \\ u_t \end{bmatrix} \right)^T \begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix} \begin{bmatrix} \phi_r \\ \phi_s \\ \phi_t \end{bmatrix} \\
&= \widehat{\nabla}^T u \begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix}^T \begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix} \widehat{\nabla} \phi,
\end{aligned}
$$

where $\widehat{\nabla}$ is the gradient operator with respect to the reference space dimensions r, s, t. With the inner-product decompiling seen in, using the local nodes we are able to precompute the symmetric geometric factors, $G_n^e = \widehat{G}_n^{e\,T} \widehat{G}_n^e$, where

$$
\widehat{G}_n^e = \begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix},
$$

for each element e and local node n. Although the geometric factors are equivalent for all nodes in the same element due to the affine reference element mapping, our implementation of the spectral element method allows for curvilinear elements; therefore,

geometric factors are distributed for each node.

Using quadrature to numerically evaluate part of the local stiffness operator on an element results in

$$\begin{aligned}(\nabla u, \nabla \phi^e_{abc})_{D^e} &= \sum_{i,j,k=0}^{N} w_{ijk} u^e_{ijk} \Big(\nabla \phi^e_{ijk} \nabla \phi^e_{abc} \Big) \\ &= \sum_{i,j,k=0}^{N} w_{ijk} u^e_{ijk} \Big(\widehat{\nabla} \phi^e_{ijk}(r_i, s_j, t_k) G^e_{ijk} \widehat{\nabla} \phi^e_{abc}(r_i, s_j, t_k) \Big).\end{aligned}$$

However,

$$\widehat{\nabla} \phi^e_{abc}(r_i, s_j, t_k) = \begin{bmatrix} \phi'_a(r_i)\delta_{bj}\delta_{ck} \\ \phi'_b(s_j)\delta_{ai}\delta_{ck} \\ \phi'_c(r_k)\delta_{ai}\delta_{bj} \end{bmatrix},$$

which vanishes if none of the indices match (i.e. $a \neq i$, $b \neq j$ and $c \neq k$). Hence reducing the computations to

$$\begin{aligned}(\nabla u, \nabla \phi^e_{abc})_{D^e} = \sum_{i=0}^{N} \Bigg(& w_{\underline{\mathbf{i}}bc} u^e_{\underline{\mathbf{i}}bc} \Big(\nabla \phi^{e^T}_{\underline{\mathbf{i}}bc} \nabla \phi^e_{abc} \Big) \\ & + w_{a\underline{\mathbf{i}}c} u^e_{a\underline{\mathbf{i}}c} \Big(\nabla \phi^{e^T}_{a\underline{\mathbf{i}}c} \nabla \phi^e_{abc} \Big) \\ & + w_{ab\underline{\mathbf{i}}} u^e_{ab\underline{\mathbf{i}}} \Big(\nabla \phi^{e^T}_{ab\underline{\mathbf{i}}} \nabla \phi^e_{abc} \Big) \Bigg),\end{aligned} \tag{5.5}$$

with the iterative index in bold and underlined. The reduction in computations required at the element level dropped from N^3 quadrature samples to $3N$ quadrature samples. Referencing back to equation (5.5), computing a quadrature sample of

$\widehat{\nabla}\phi^e_{abc}$ also uses a linear $3N$ amount of nodal evaluations. Without using quadrature nodes as the points in the Lagrange basis, $\widehat{\nabla}\phi^e_{abc}$ would require N^3 nodal evaluations. To summarize, the correlation between the Lagrange points and quadrature reduced computations of the $(u, \phi)_{D^e}$ from $O(N^6)$ to $O(N^2)$.

5.3 Implementation

In this section, I combine the simplifications in the numerical discretizations seen in the previous section with the OCCA programming model from chapter 3 to develop a parallel algorithm suited for multi-core and many-core architectures. There have been different approaches to parallelizing the Navier-Stokes equations using distributed nodes ([45], [27]) as well as GPU parallelism ([25], [24], [43], [7]). I first discuss the operations for applying advection, followed by the elliptic operators that arise from diffusion and pressure.

5.3.1 Algorithm Walkthrough

The same partition of work is applied in all update steps, where a work-group manages the update of a single element, done in conjunction by its work-items. Using the same sweeping method introduced for the finite difference implementation in chapter 4, the work-group is arranged as a two dimensional block which sweeps through the third dimension. figure 5.2 demonstrates the parallelism chosen for the work-items in each

work-group.

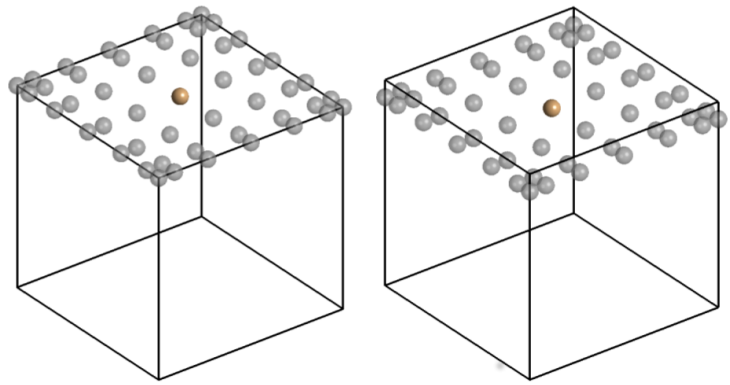


Figure 5.2 : In each sweep, the work-group iterates one point in the sweeping direction, t.

The assumption of each element using locally high-order test and trial spaces places a limit on the dimension of each work-group, where the limit of work-items on a work-group is 256 or 512 in OpenCL and CUDA platforms. Reducing the work-group to a two dimensional block allows for a high-order scheme to be implemented on GPUs efficiently. Further, the memory model in [37] which makes use of shared memory for storing two dimensions of data and registers for the sweeping direction can similarly be applied for spectral element methods. By using tensor-product Lagrange basis for numerical integration, computations to apply the stiffness operator after simplifications in equation (5.5) reduce to a stencil-like operations.

Referring back to equation (5.5), the numerical integration for a specific trial function ϕ^e_{abc} uses nodal contributions from

$$RST_{abc} = \left(\bigcup_{j=0}^{N} (r_j, s_b, t_c)\right) \cup \left(\bigcup_{j=0}^{N} (r_a, s_j, t_c)\right) \cup \left(\bigcup_{j=0}^{N} (r_a, s_b, t_j)\right),$$

expanding equation (5.5) to

$$\begin{aligned}
(\nabla u, \nabla \phi^e_{abc})_{D^e} &= \sum_{i=0}^{N} \Bigg(\quad w_{\underline{\mathbf{i}}bc} u^e_{\underline{\mathbf{i}}bc} \Big(\nabla \phi^e_{\underline{\mathbf{i}}bc}{}^T \nabla \phi^e_{abc} \Big) \\
&\quad + \quad w_{a\underline{\mathbf{i}}c} u^e_{a\underline{\mathbf{i}}c} \Big(\nabla \phi^e_{a\underline{\mathbf{i}}c}{}^T \nabla \phi^e_{abc} \Big) \\
&\quad + \quad w_{ab\underline{\mathbf{i}}} u^e_{ab\underline{\mathbf{i}}} \Big(\nabla \phi^e_{ab\underline{\mathbf{i}}}{}^T \nabla \phi^e_{abc} \Big) \Bigg) \\
&= \sum_{j=0}^{N} \Bigg(\quad \Big(\textstyle\sum_{i=0}^{N} w_{\underline{\mathbf{i}}bc} u^e_{\underline{\mathbf{i}}bc} \nabla \phi^e_{\underline{\mathbf{i}}bc}(r_{\underline{\mathbf{j}}}, s_b, t_c) \Big)^T \nabla \phi^e_{abc}(r_{\underline{\mathbf{j}}}, s_b, t_c) \\
&\quad + \quad \Big(\textstyle\sum_{i=0}^{N} w_{a\underline{\mathbf{i}}c} u^e_{a\underline{\mathbf{i}}c} \nabla \phi^e_{a\underline{\mathbf{i}}c}(r_a, s_{\underline{\mathbf{j}}}, t_c) \Big)^T \nabla \phi^e_{abc}(r_a, s_{\underline{\mathbf{j}}}, t_c) \\
&\quad + \quad \Big(\textstyle\sum_{i=0}^{N} w_{ab\underline{\mathbf{i}}} u^e_{ab\underline{\mathbf{i}}} \nabla \phi^e_{ab\underline{\mathbf{i}}}(r_a, s_b, t_{\underline{\mathbf{j}}}) \Big)^T \nabla \phi^e_{abc}(r_a, s_b, t_{\underline{\mathbf{j}}}) \Bigg)
\end{aligned} \tag{5.6}$$

Taking the work-group e and one local work-item `(li,lj)`, the particular work-item is responsible for updating $u^e_{(li,lj,k)}$ as the work-group sweeps in the t direction. Fixing k, the work-item (li, lj) calculates $\widehat{\nabla} u^e_{(li,lj,k)}$ only from contributions of entries in

$RST_{(li,lj,k)}$, resulting in

$$\begin{bmatrix} \sum_{i=0}^{N} u^e_{(\underline{\mathbf{i}},lj,k)} \phi'_{li}(r_{\underline{\mathbf{i}}}) \\ \sum_{i=0}^{N} u^e_{(li,\underline{\mathbf{i}},k)} \phi'_{lj}(r_{\underline{\mathbf{i}}}) \\ \sum_{i=0}^{N} u^e_{(li,lj,\underline{\mathbf{i}})} \phi'_{k}(r_{\underline{\mathbf{i}}}) \end{bmatrix},$$

with multiplies the result by the pre-computed geometric factors. Rather than multiplying by

$$G^e_{ijk} \;=\; \widehat{G}^e_{ijk}{}^T \widehat{G}^e_{ijk},$$

the quadrature weights are pre-applied by instead storing

$$G^e_{ijk} \;=\; \widehat{G}^e_{ijk}{}^T \begin{bmatrix} w_i & 0 & 0 \\ 0 & w_j & 0 \\ 0 & 0 & w_k \end{bmatrix} \widehat{G}^e_{ijk},$$

Resulting is a three dimensional vector containing the evaluation of

$$Gu^e_{(li,lj,k)} = \underbrace{\begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix}^T \begin{bmatrix} w_i & 0 & 0 \\ 0 & w_j & 0 \\ 0 & 0 & w_k \end{bmatrix} \begin{bmatrix} x_r & x_s & x_t \\ y_r & y_s & y_t \\ z_r & z_s & z_t \end{bmatrix}}_{G^e_{(li,lj,k)}} \widehat{\nabla} u^e_{(li,lj,k)}.$$

A partial update for the solution $u^e_{(li,lj,k)}$ in the r and s directions comes from

$$\begin{aligned} u^e_{(li,lj,k)} \quad \leftarrow \quad & \sum_{j=0}^{N} Gur^T_{(\underline{\mathbf{j}},lj,k)} \nabla \phi^e_{abc}(r_{\underline{\mathbf{j}}}, s_b, t_c) \\ + \quad & \sum_{j=0}^{N} Gus^T_{(li,\underline{\mathbf{j}},k)} \nabla \phi^e_{abc}(r_a, s_{\underline{\mathbf{j}}}, t_c), \end{aligned}$$

and the incomplete operator for all nodes in the sweeping (t) direction is updated

$$Lut_{\underline{\mathbf{i}}} \texttt{ += } Gut^T_{(li,lj,k)} \nabla \phi^e_{abc}(r_a, s_b, t_{\underline{\mathbf{i}}}), \text{ for } i \in Z^N.$$

After all sweeps, the full stiffness operator is obtained by adding contributions from Lut,

$$u^e_{(li,lj,\underline{\mathbf{i}})} \texttt{ += } Lut_{\underline{\mathbf{i}}}, \text{ for } i \in Z^N.$$

5.3.2 Memory Layout

Taken a similar approach as [37], the work-group stores plane data in shared memory and the data in the sweeping direction in register memory. I give a visual representation of the memory pipeline in the given algorithm and describe in detail each step. To simplify the parallel algorithm for the reader, I write the memory layout in the viewpoint of a fixed work-item as the work-group sweeps in the t direction.

First I fix the polynomial order of the scheme to be of fourth order ($N = 4$) for simplicity. Taking an arbitrary element e, we'll look at the center work-item $(2, 2)$

and it's memory scope throughout the algorithm.

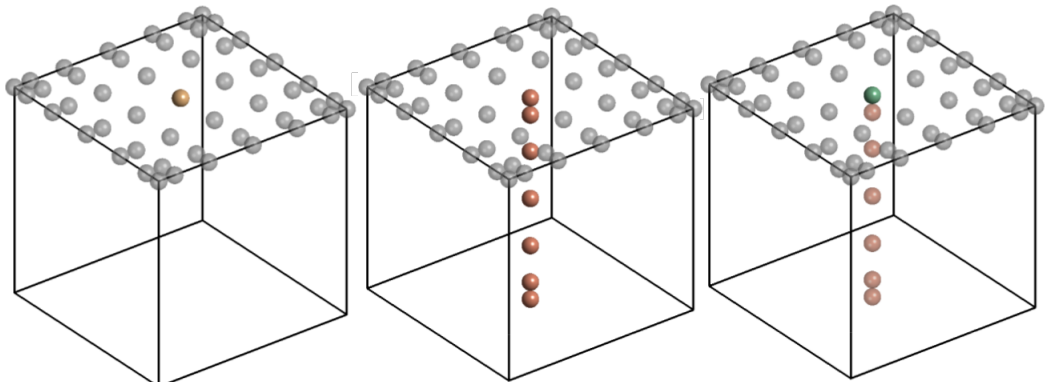


Figure 5.3 : *Left-to-Right:* The highlighted node corresponds to the work-item (2,2). The work-item loads in a register array in the sweeping direction. During the first sweep, the data value in the current sweep is stored in shared memory.

Prior to sweeping over the t direction, our work-item loads values $u^e_{2,2,k}$ in a register array, `uk[]`, as seen in figure 5.3, for it is responsible of updating those solution values. Starting the sweep, the work-group has its third index, k, fixed to 0 which will iterate as $k = 0, 1, 2, 3, 4$. Our work-item now stores `uk[0]` in the shared memory array `Lu[2][2]` on it's local entries $(2, 2)$ (figure 5.4). However, before calling a memory fence, the values in the shared array are not guaranteed to be fully loaded (figure 5.4).

The storage of `Lu` allows each work-item to calculate $GDu = G^e_{(li,lj,0)}\widehat{\nabla}u^e(li, lj, 0)$. Two of the three entries in GDu are stored in the shared buffers `Lv` and `Lw`, while the third entry, which represents the sweeping direction, is stored in private memory. Following the same steps as figure 5.4, figure 5.5 uses the same stencil format to

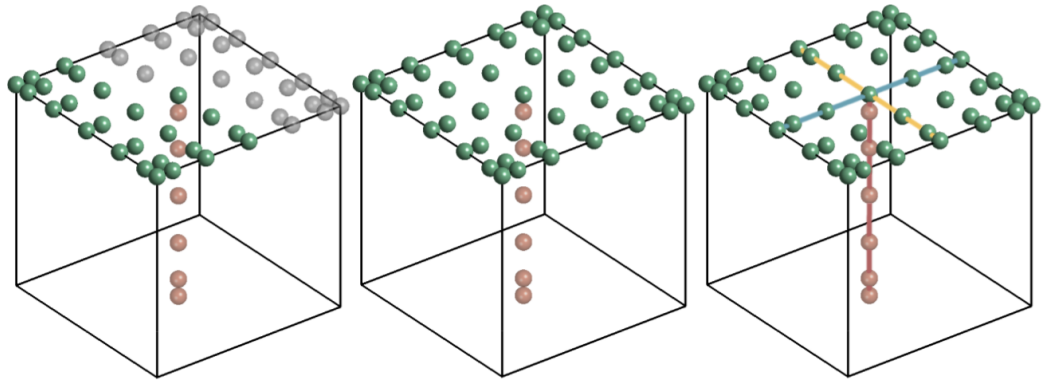


Figure 5.4 : *Left-to-Right:* The value of $u^e(2,2,0)$ is stored in `Lu`. Before calling for a memory fence, the memory is undefined in the local array. Calling for a memory fence gives our work-item access to all parallel reads stored in shared memory.

calculate $\widehat{\nabla}\phi^e_{abc}GDu$.

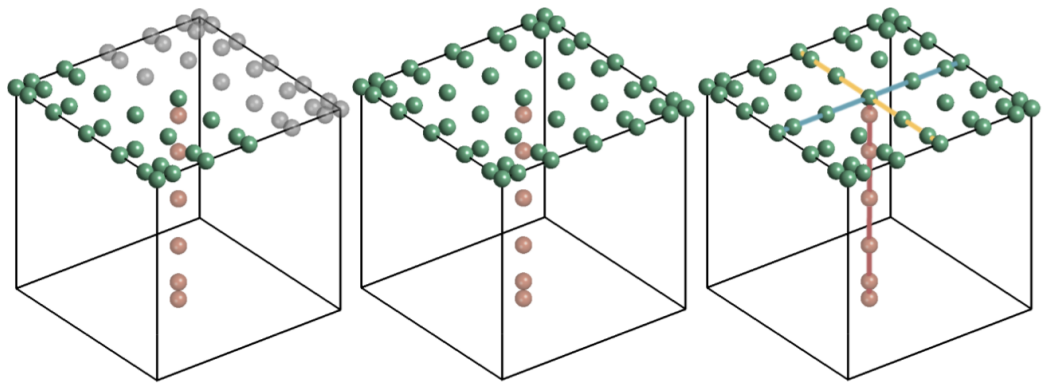


Figure 5.5 : *Left-to-Right:* The value of $G^e_{(2,2,0)}\nabla u^e(2,2,0)$ is stored in a register and shared arrays, `Lv`, `Lw`. Before calling for a memory fence, the memory is undefined in the local array. Calling for a memory fence gives our work-item access to all parallel reads stored in shared memory.

Adding a memory fence guarantees concurrent data is synchronized (figure 5.5). After calculating each work-item finishes the stiffness operator update for `(li,lj)`, the whole process is repeated for each sweep until the last plane of nodes (figure 5.6).

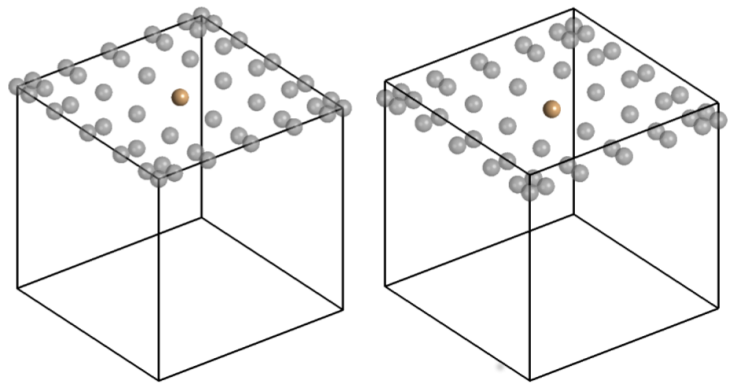


Figure 5.6 : In each sweep, the work-group iterates one point in the sweeping direction, t.

5.4 Performance Results

This section displays performance results for the kernel responsible of applying the simplified discrete operator seen in equation (5.6). Computational work (GFLOPS) and data bandwidth (GB/s) are used to display two distinct performance metrics in the following figures. Different polynomial orders are displayed in the performance figures due to the increase in the FLOP to data read ratio.

Initial performance results on the Intel i7-3930k is shown in figure 5.7. The performance is similar across the supported languages but an outlier can be quickly detected for polynomial order 7, where the work-group contains 8^2 work-items (a multiple of eight). Although not shown for GNEK, chapter 6 contains a kernel which exploits the vectorization for work-groups with a multiple of 8 number of work-items. Modifying the work-group arrangement seems to overly affect performance on multi-core

CPU architectures. By restricting programmers to the work-group/work-item programming model, compiler architects can plan on effective optimization techniques compared with a function compiled for an all-purpose processor. OpenCL shows to be highly tuned for the work-group/work-item programming model which can expose vectorizations.

Similar to the results from figure 5.7 can be seen in [2], in which OpenCL managed to perform similarly to OpenMP and Intel's thread building blocks (TBB). The performance between the Intel and AMD OpenCL platforms also coincided with [2], where Intel outperformed the AMD platform. Hence, it can be implied that Intel's OpenCL platform contains architecture-dependent optimizations.

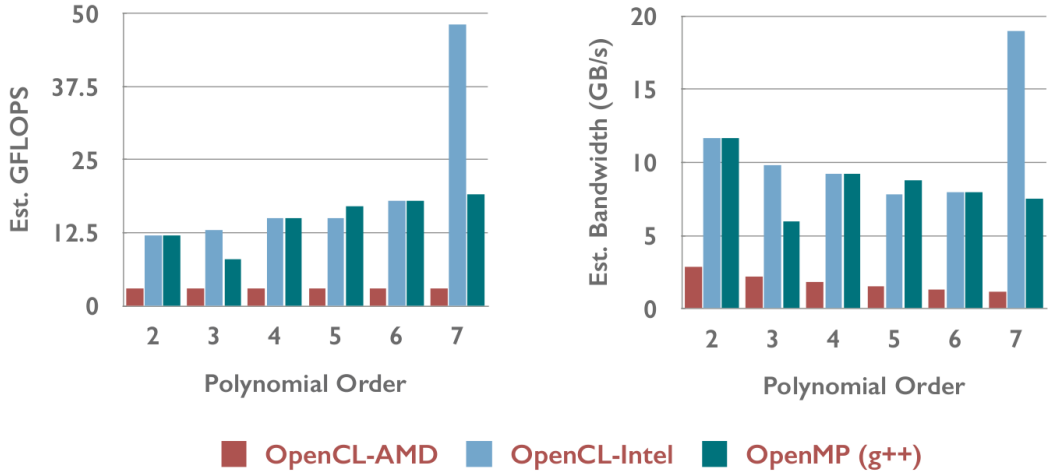


Figure 5.7 : Performance and bandwidth comparisons of the VTI volume kernel are given for the different languages available on an Intel i7-3930k (x86 architecture). Note the peak performance for polynomial order 5 is do to the work-group arrangement becoming a multiple of 8 and thus properly vectorizing the kernel.

Displayed in figure 5.8 is the performance and bandwidth results for the operator kernel described in equation (5.6) on high-end consumer AMD and NVIDIA cards. One processor in the dual-processor Radeon 7990 (Tahiti) and NVIDIA's Titan are used in this performance study. In this case, CUDA and OpenCL on the NVIDIA Titan card have comparable performance.

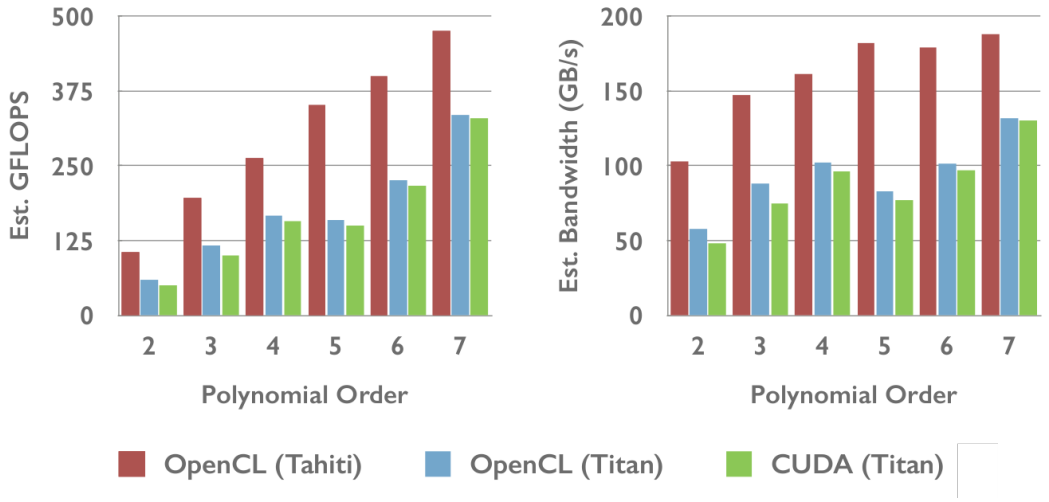


Figure 5.8 : Performance and bandwidth comparisons of the VTI volume kernel are given for the different languages available on a Radeon 7970 (AMD Tahiti) and an NVIDIA Titan card.

CHAPTER 6

RiDG

In this chapter, I focus on RiDG, a discontinuous Galerkin (DG) algorithm implementation using OCCA for seismic wave propagations. I first discuss the distinct partial differential equations solved in RiDG and their generalization in the conservative form. Following is the discretization and formulation of the DG operators which guides the parallel algorithms suited for multi-core and many-core architectures. Lastly, I give performance results on the different architectures supported in OCCA.

6.1 Introduction

The RiDG project offers three distinct velocity models that describe different forms of material anisotropy. The first model that is discussed is the linearized acoustic wave equations in isotropic media. The second model adds linear elasticity with a

plane of symmetry, vertical transverse isotropy (VTI) model. The VTI model assumes symmetry along an axis, in our case given by z, or the depth direction. The third model, tilted transverse isotropy (TTI), adds extra parameters to the VTI equations that define the axis of symmetry for the transverse isotropy model.

6.1.1 Acoustic Wave Equation

I start with one of the simplest forms of modeling wave propagations is given by the acoustic wave equation. The equations assume a homogeneous isotropy model where the wave-field has no preferred direction of propagation. The equations are given by

$$p_{tt} = c^2 \Delta p,$$

where p is defined as acoustic pressure in space and time.

The first-order formulation is given by the system of equations

$$\begin{aligned} p_t &= -c^2(u_x + v_y + w_z) \\ u_t &= -p_x \\ v_t &= -p_y \\ w_t &= -p_z, \end{aligned}$$

where p is the pressure field and the vector (u, v, w) denotes the particle velocity components. In a compressed matrix form, which is later used to generalize the

different isotropy models, the first-order equations are represented by

$$q_t = -\nabla \cdot F(q),$$

where

$$q = \begin{bmatrix} p \\ u \\ v \\ w \end{bmatrix}, \; F = \left(\hat{A}_1 e_1 + \hat{A}_2 e_2 + \hat{A}_3 e_3\right)$$

$$\hat{A}_1 = \begin{pmatrix} 0 & c^2 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \; \hat{A}_2 = \begin{pmatrix} 0 & 0 & c^2 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \; \hat{A}_3 = \begin{pmatrix} 0 & 0 & 0 & c^2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

I abuse notation when writing $\nabla \cdot F(q)$ to mean

$$\begin{aligned} \nabla \cdot F(q) &= \nabla \cdot \left(\hat{A}_1 q e_1 + \hat{A}_2 q e_2 + \hat{A}_3 q e_3\right) \\ &= \frac{\partial \hat{A}_1 q}{\partial x} + \frac{\partial \hat{A}_2 q}{\partial y} + \frac{\partial \hat{A}_3 q}{\partial z} \end{aligned}$$

6.1.2 Vertical Transverse Isotropy

Adding anisotropy symmetrically along the z axis defines vertical transverse isotropy (VTI). Using the VTI equations offered by Duveneck et al [15], the system of equations

for the wave equations become

$$
\begin{aligned}
\sigma_t^H &= \rho c^2 \Big[(1+2\epsilon)(u_x + v_y) + (\sqrt{1+2\sigma}) w_z \Big] \\
\sigma_t^V &= \rho c^2 \Big[(\sqrt{1+2\delta})(u_x + v_y) + w_z \Big] \\
u_t &= \rho^{-1} \sigma_x^H \\
v_t &= \rho^{-1} \sigma_y^H \\
w_t &= \rho^{-1} \sigma_z^V
\end{aligned}
\tag{6.1}
$$

where, similar to the full isotropy case, p is the pressure field and the vector (u, v, w) denotes the particle velocity components. An addition of horizontal and vertical stress components, σ^H and σ^V, as well as the Thomson parameters ϵ and δ are added to the VTI equations as well. In the same compressed matrix form given for the acoustic wave equation, the first-order equations for the VTI Duvuneck model are given by

$$q_t = -\nabla \cdot F(q),$$

where

$$q = \begin{bmatrix} \sigma^H \\ \sigma^V \\ u \\ v \\ w \end{bmatrix}, \; F = \hat{W}^{-1} \Big(\hat{A}_1 e_1 + \hat{A}_2 e_2 + \hat{A}_3 e_3 \Big),$$

$$\hat{W}^{-1} = \begin{pmatrix} \rho c^2(1+2\epsilon) & \rho c^2\sqrt{1+2\delta} & 0 & 0 & 0 \\ \rho c^2\sqrt{1+2\delta} & \rho c^2 & 0 & 0 & 0 \\ 0 & 0 & \rho^{-1} & 0 & 0 \\ 0 & 0 & 0 & \rho^{-1} & 0 \\ 0 & 0 & 0 & 0 & \rho^{-1} \end{pmatrix},$$

$$\hat{A}_1 = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \ \hat{A}_2 = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \ \hat{A}_3 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}.$$

6.1.3 Tilted Transverse Isotropy

Generalizing the VTI equations to add anisotropy symmetrically along an arbitrary axis defines tilted transverse isotropy (TTI). The resulting system of equations are given by

$$\begin{aligned} \sigma_t^H &= \rho c^2\Big[(1+2\epsilon)A_{ij} + (\sqrt{1+2\sigma})B_{ij}\Big]\frac{\partial \vec{v}_i}{\partial x_j} \\ \sigma_t^V &= \rho c^2\Big[(\sqrt{1+2\delta})A_{ij} + B_{ij}\Big]\frac{\partial \vec{v}_i}{\partial x_j} \\ \frac{\partial \vec{v}_i}{\partial t} &= \rho^{-1}\frac{\partial}{\partial x_j}\Big[A_{ij}\sigma^H + B_{ij}\sigma^V\Big], \end{aligned}$$

where, in addition to the VTI parameters, the matrix of coefficients $A, B \in \mathbb{R}^{3\times 3}$ come from the stress tensor C_{ijkl}.

In the same compressed matrix form given for the acoustic wave equation, the first-

order equations for the TTI model are given by

$$q_t = -\nabla \cdot F(q),$$

where

$$q = \begin{bmatrix} \sigma^H \\ \sigma^V \\ u \\ v \\ w \end{bmatrix}, \; F = \hat{W}^{-1}\Big(\hat{A}_1 e_1 + \hat{A}_2 e_2 + \hat{A}_3 e_3\Big),$$

$$\hat{W}^{-1} = \begin{pmatrix} \rho c^2(1+2\epsilon) & \rho c^2\sqrt{1+2\delta} & 0 & 0 & 0 \\ \rho c^2\sqrt{1+2\delta} & \rho c^2 & 0 & 0 & 0 \\ 0 & 0 & \rho^{-1} & 0 & 0 \\ 0 & 0 & 0 & \rho^{-1} & 0 \\ 0 & 0 & 0 & 0 & \rho^{-1} \end{pmatrix},$$

$$\hat{A}_i = \begin{pmatrix} 0 & 0 & - & A_i^T & - \\ 0 & 0 & - & B_i^T & - \\ | & | & 0 & 0 & 0 \\ A_i & B_i & 0 & 0 & 0 \\ | & | & 0 & 0 & 0 \end{pmatrix} \text{ for } i = 1, 2, 3.$$

6.2 Discretization

Similar to the spectral element method in chapter 5, it is assumed the domain $\Omega \in \mathbb{R}^3$ is partitioned into non-overlapping and conforming elements ($\Omega \approx \Omega_h = \cup_e D^e$); however, for this example, tetrahedral elements are used. Let the test and trial

function spaces both notated by V, a piecewise polynomial space of degree N that is C^0-continuous across elements. The general form for the wave equation under different velocity models is given by

$$q_t = -\nabla \cdot F(q),$$

and is expressed in the following variational form

$$(q_t, \phi)_{\Omega_h} = -\Big(\nabla \cdot F(q), \phi\Big)_{\Omega_h} + \Big(G^*(q) \cdot \vec{n}, \phi\Big)_{\partial\Omega_h} \quad \forall \phi \in V,$$

where the numerical flux is given by G^*.

For the three models described in the previous section, the pseudo-upwinding scheme

$$G^*(q) = (D + \tau D^T D)\{q\},$$

is used, with $\tau > 0$ used as a stabilization parameter. The respective formulation of D for the acoustic, VTI and TTI equations are given by

$$D_{acoustic} = \begin{pmatrix} 0 & n_x & n_y & n_z \\ n_x & 0 & 0 & 0 \\ n_y & 0 & 0 & 0 \\ n_z & 0 & 0 & 0 \end{pmatrix}, \; D_{VTI} = D_{TTI} = \begin{pmatrix} 0 & 0 & n_x & n_y & 0 \\ 0 & 0 & 0 & 0 & n_z \\ n_x & 0 & 0 & 0 & 0 \\ n_y & 0 & 0 & 0 & 0 \\ 0 & n_z & 0 & 0 & 0 \end{pmatrix}.$$

6.3 Implementation

Another similar trait to the spectral element method in GNEK is the partition of work at the element-level. Using the natural element decoupling from the discontinuous Galerkin method, asynchronous calculations appear between volume and surface terms,

$$(q_t, \phi)_{D^e} = \underbrace{-\Big(\nabla \cdot F(q), \phi\Big)_{D^e}}_{\text{Volume Term}} + \underbrace{\Big(G^*(q) \cdot \vec{n}, \phi\Big)_{\partial D^e}}_{\text{Surface Term}} \quad \forall \phi \in V(D^e), \tag{6.2}$$

where the only coupling between elements derives from the numerical scheme integrating across element interfaces as seen in figure 6.1 for the 2D case.

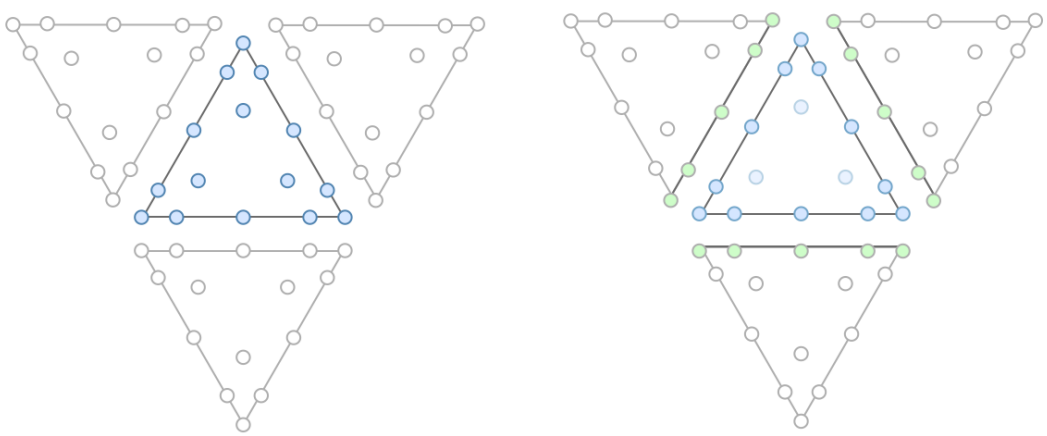


Figure 6.1 : The figure shows an arbitrary 2D triangular element and its neighbors. The left figure shows the element dependency when calculating the volume term, which in this case is the element itself. The right figure shows the element dependency when calculating the surface term, which requires neighboring surface values to calculate the numerical flux.

This section goes through the volume term kernel in the discontinuous Galerkin update. To relate with the example in chapter 4, I choose to go over the VTI volume kernel. Following the implementation and included pseudocode, the chapter ends with performance results on the different architectures and languages which OCCA supports.

Using the simplifications found in [30], the set of equations in 6.1 reduce to

$$\begin{aligned}
\frac{\partial q}{\partial t} &= -M^{-1}\left(S_x A_1 q + S_y A_2 q + S_z A_3 q\right) + LIFT((G\cdot\vec{n})^* - (G\cdot\vec{n})^-) \\
&= \begin{bmatrix} Dr, & Ds, & Dt \end{bmatrix} \left(\begin{bmatrix} r_x^e \\ s_x^e \\ t_x^e \end{bmatrix} A_1 q + \begin{bmatrix} r_y^e \\ s_y^e \\ t_y^e \end{bmatrix} A_2 q + \begin{bmatrix} r_z^e \\ s_z^e \\ t_z^e \end{bmatrix} A_3 q \right) \\
&\quad + LIFT((G\cdot\vec{n})^* - (G\cdot\vec{n})^-).
\end{aligned}$$

The Jacobian for the reference mapping is given by

$$\begin{bmatrix} r_x^e & s_x^e & t_x^e \\ r_y^e & s_y^e & t_y^e \\ r_z^e & s_z^e & t_z^e, \end{bmatrix}$$

which are precomputed for each element e. However, the matrices Dr, Ds, and Dt are independent of the element and thus can be reused for each element.

The pseudocode is given by:

```
for each work-group e:
  for each work-item n:
    if(n < 9)
      s_geo[n] = g_geo[9*e + n];

    s_sigmaH[n] = g_Q[e*Np*fields + 0*Np + n]; // Fields = 5 in this case
    s_sigmaV[n] = g_Q[e*Np*fields + 1*Np + n];
    s_u[n]      = g_Q[e*Np*fields + 2*Np + n];
    s_v[n]      = g_Q[e*Np*fields + 3*Np + n];
    s_w[n]      = g_Q[e*Np*fields + 4*Np + n];

    barrier(localMemFence); // Make sure shared memory is in sync

    for(int m = 0; m < Np; ++m){
      // Define:
      //   [rx,sx,tx] = s_geo[0,1,2]
      //   [ry,sy,ty] = s_geo[3,4,5]
      //   [rz,sz,tz] = s_geo[6,7,8]

      p_dx = rx*g_Dr[m][n] + sx*g_Ds[m][n] + tx*g_Dt[m][n];
      p_dy = ry*g_Dr[m][n] + sy*g_Ds[m][n] + ty*g_Dt[m][n];
      p_dz = rz*g_Dr[m][n] + sz*g_Ds[m][n] + tz*g_Dt[m][n];

      p_sigmaHx += p_dx*s_sigmaH[m];
      p_sigmaHy += p_dy*s_sigmaH[m];
      p_sigmaVz += p_dz*s_sigmaV[m];
      p_ux       += p_dx*s_u[m];
      p_vy       += p_dx*s_v[m];
      p_wz       += p_dx*s_w[m];
    }

    p_rho   = g_rho[e*Np + n];   // ρ
    p_c2    = g_c2[e*Np + n];    // c^2
    p_1a2e  = g_1a2e[e*Np + n];  // 1 + 2ε
    p_s1a2d = g_s1a2d[e*Np + n;  // √(1 + 2δ)

    p_rhoc2  = p_rho*p_c2;
    p_rhoInv = 1.0/p_rho;

    Q2[e*Np*fields + 0*Np + n] = p_rhoc2*( p_1a2e(p_ux + p_vy) + p_s1a2d*p_wz);
    Q2[e*Np*fields + 1*Np + n] = p_rhoc2*(p_s1a2d(p_ux + p_vy) +         p_wz);
    Q2[e*Np*fields + 2*Np + n] = p_rhoInv*p_sigmaHx;
    Q2[e*Np*fields + 3*Np + n] = p_rhoInv*p_sigmaHy;
    Q2[e*Np*fields + 4*Np + n] = p_rhoInv*p_sigmaHz;
  end for
end for
```

Listing 6.1: Discontinuous Galerkin volume operator pseudocode

6.4 Performance Results

I choose the VTI volume kernel as a representative for the performance seen using OCCA across CPU and GPU architectures. Computational work (GFLOPS) and data bandwidth (GB/s) are used to display two distinct performance metrics in the following figures. Different polynomial orders are displayed in the performance figures due to the increase in the FLOP to data read ratio.

Initial performance results on the Intel i7-3930k is shown in figure 6.2. The performance is similar across the supported languages but an outlier can be quickly detected for polynomial order 5, where the work-group contains 56 work-items (a multiple of eight).

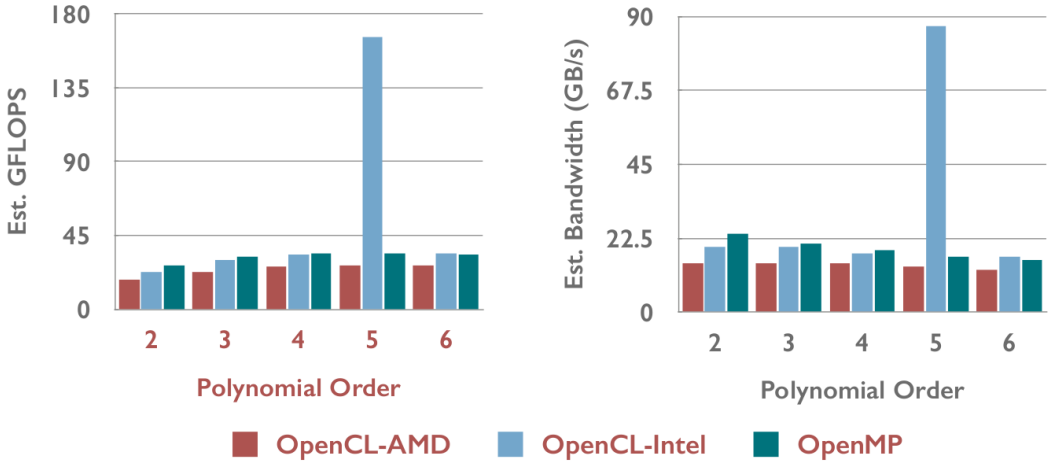


Figure 6.2 : Performance and bandwidth comparisons of the VTI volume kernel are given for the different languages available on an Intel i7-3930k (x86 architecture). Note the peak performance for polynomial order 5 is do to the work-group arrangement becoming a multiple of 8 and thus properly vectorizing the kernel.

Going back to the CPU architecture optimizations discussed in chapter 2, an increase in performance for a power of 2 can usually be explained by vectorization. The approach taken to make use of vectorization was to artificially insert work-items to create a multiple of 8 work-group arrangement and allowing the supplementary work-items to work on dummy data. A more consistent performance growth can be seen in figure 6.3. By manually changing work-group arrangements, a significant performance gain is achieved across languages; thus, auto-tuning work-group arrangements is a possible future feature for OCCA to support.

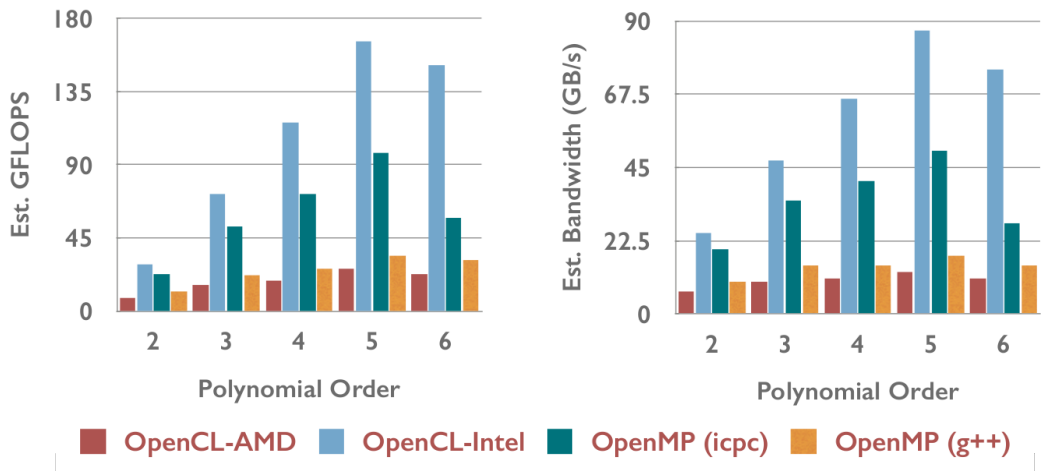


Figure 6.3 : Performance and bandwidth comparisons of the VTI volume kernel are given for the different languages available on an Intel i7-3930k (x86 architecture). The difference between this figure and figure 6.2 is the work-group arrangement. The number of work-items was forced to be a multiple of 8, where the supplementary work-items work on dummy data.

Displayed in figure 6.4 is the performance and bandwidth results for the VTI kernel on high-end consumer AMD and NVIDIA cards. One processor in the dual-processor Radeon 7990 (Tahiti) and NVIDIA's Titan are used in this performance study. Note

that with NVIDIA having the capability to exploit their own hardware and same precursor compiler as OpenCL, it is natural for CUDA to outperform OpenCL. Without bias, the programmer would choose CUDA for computing on the Titan in this case and OpenCL on the Tahiti card.

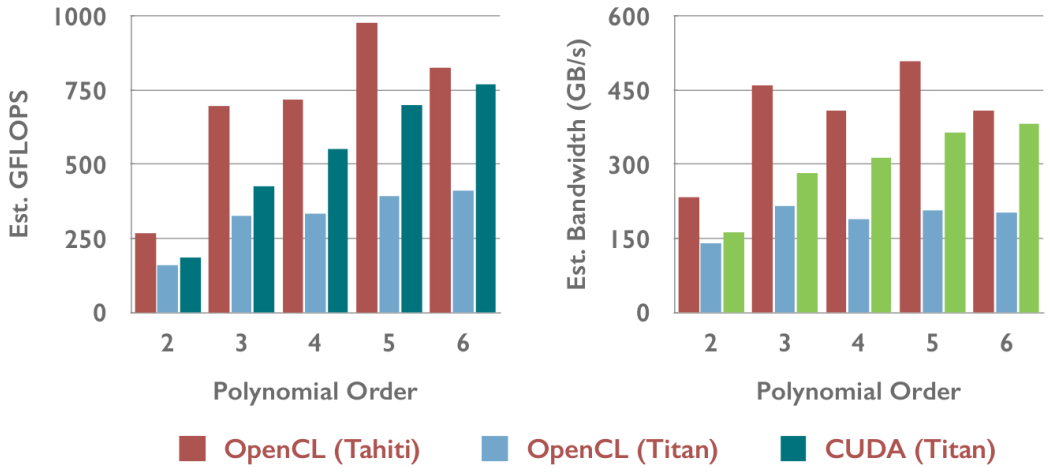


Figure 6.4 : Performance and bandwidth comparisons of the VTI volume kernel are given for the different languages available on a Radeon 7970 (AMD Tahiti) and an NVIDIA Titan card.

CHAPTER 7

Conclusions and Future Work

There were two goals in this thesis, one was to develop a flexible and extensible framework that simplifies heterogeneous programming without sacrificing performance. The second objective in the thesis work was to show successful applications of this programming framework on a spectrum of different numerical method implementations.

7.1 Conclusion

I reviewed past approaches that focused on the translation between different languages, such as native code transformation, intermediate assembly code transformation and compiler-based approaches. Although there is a plethora of procedures aimed at automating code conversion, the novelty of OCCA comes from its lightweight macro-based language that expands to supported languages by covering major lan-

guage keywords. Together with the host API, OCCA targets different architectures with an offloading model to overlap the supported C/C++ API. With the use of just-in-time compilation and the custom macro-based language, OCCA enables the use of different programming languages on multiple architectures. The overlap of languages that target the same architecture, such as OpenMP and OpenCL for CPUs or OpenCL and CUDA for GPUs, gives the user the ability to choose the top-performing language for the given architecture without bias.

In this thesis, distinct numerical methods and their OCCA implementations were shown alongside performance results on CPU and GPU architectures using OpenMP, OpenCL and CUDA. To exhibit understanding of numerical methods, I detail the high performance algorithms used in a finite difference method used in seismic imaging, a spectral element method used in computational fluid dynamics and a discontinuous Galerkin method also used in seismic imaging in chapter 4, chapter 5, and chapter 6 respectively. To further demonstrate the flexibility of OCCA, distinct numerical method implementations that utilize OCCA are shown in ([20], [39], [22]) and have been presented in different conferences [36],[23],[21]). Along with conferences, OCCA has been used for educational purposes at Rice University where the students gave positive feedback on the simplicity of the programming model.

I achieved the goals for this thesis with the development and testing of OCCA on various numerous projects using different numerical methods. The current OCCA framework is able to run OpenMP, OpenCL and CUDA as a serial process or in a

distributed manner with MPI. Regarding programming languages which run OCCA, our OCCA library is written in C++ so I wish to acknowledge Lucas Wilcox for creating a C and Fortran interface for OCCA.

7.2 Future Work

There is still room for improvement in OCCA which is left as future work. A major milestone set for future releases of OCCA is the development of a unified distributed programming environment which would abstract message parsing from inside device kernels through MPI calls. Smaller features that will be inspected in the future include auto-tuning kernels and kernel sizes through work-group arrangement. Additionally, OCCA-enabled hardware theoretically includes Intel's Xeon Phi through OpenMP and OpenCL as well as FPGAs through OpenCL and it is of interest to pursue performance studies with these two distinct architectures.

APPENDIX A

Appendix: OCCA Kernel Keywords

Provided are OCCA keywords categorized in tables by their use or purpose. Each table provides similar OCCA keywords on the left-most column. Adjacent to the OCCA keyword column is the OpenMP macro expansion, followed by the OpenCL macro expansion and the CUDA macro expansion on the right-most column.

OCCA	OpenMP	OpenCL	CUDA
occaInnerId0		get_local_id(0)	threadIdx.x
occaInnerId1		get_local_id(1)	threadIdx.y
occaInnerId2		get_local_id(2)	threadIdx.z
occaOuterId0		get_group_id(0)	blockIdx.x
occaOuterId1		get_group_id(1)	blockIdx.y
occaGlobalId0	occaInnerId0 + occaInnerDim0*occaOuterId0	get_global_id(0)	threadIdx.x + blockIdx.x*blockDim.x
occaGlobalId1	occaInnerId1 + occaInnerDim1*occaOuterId1	get_global_id(1)	threadIdx.y + blockIdx.y*blockDim.y
occaGlobalId2	occaInnerId2	get_global_id(2)	threadIdx.z

Table A.1 : OCCA keywords used to obtain a scope's work-group [[block]] and work-item [[thread]] ID.

OCCA	OpenMP	OpenCL	CUDA
occaInnerDim0	occaDims[0]	get_local_size(0)	blockDim.x
occaInnerDim1	occaDims[1]	get_local_size(1)	blockDim.y
occaInnerDim2	occaDims[2]	get_local_size(2)	blockDim.z
occaOuterDim0	occaDims[3]	get_num_groups(0)	gridDim.x
occaOuterDim1	occaDims[4]	get_num_groups(1)	gridDim.y
occaGlobalDim0	occaInnerDim0*occaOuterDim0	get_global_size(0)	occaInnerDim0*occaOuterDim0
occaGlobalDim1	occaInnerDim1*occaOuterDim1	get_global_size(1)	occaInnerDim1*occaOuterDim1
occaGlobalDim2	occaInnerDim2*occaOuterDim2	get_global_size(2)	occaInnerDim2

Table A.2 : OCCA keywords related to storing work-group [[block]] and work-item [[thread]] sizes.

OCCA	OpenMP	OpenCL	CUDA
occaInnerFor	occaInnerFor2 occaInnerFor1 occaInnerFor0		
occaInnerFor0	for(occaInnerId0 = 0; occaInnerId0 < occaInnerDim0; ++occaInnerId0)		
occaInnerFor1	for(occaInnerId1 = 0; occaInnerId1 < occaInnerDim1; ++occaInnerId1)		
occaInnerFor2	for(occaInnerId2 = 0; occaInnerId2 < occaInnerDim2; ++occaInnerId2)		
occaOuterFor0	for(occaOuterId0 = 0; occaOuterId0 < occaOuterDim0; ++occaOuterId0)		
occaOuterFor1	for(occaOuterId1 = 0; occaOuterId1 < occaOuterDim1; ++occaOuterId1)		
occaOuterFor2			
occaGlobalFor0	occaOuterFor0 occaInnerFor0		
occaGlobalFor1	occaOuterFor1 occaInnerFor1		
occaGlobalFor2	occaInnerFor2		

Table A.3 : OCCA keywords related to explicitly displaying work-group [[block]] and work-item [[thread]] loop scopes.

OCCA	OpenMP	OpenCL	CUDA
occaShared		__local	__shared__
occaPointer		__global	
occaConstant		__constant	__constant__
occaVariable			
occaRestrict	__restrict__	restrict	__restrict__
occaVolatile		volatile	__volatile__
occaConst	const	const	const
occaAligned	__attribute__ ((aligned (__BIGGEST_ALIGNMENT__)))		

Table A.4 : OCCA keywords related to OCCA variables attributes.

OCCA	OpenMP	OpenCL	CUDA
occaKernelInfoArg	const int *occaDims	__global int *dims	int *dims
occaFunctionInfoArg	const int *occaDims, int occaInnerId0, int occaInnerId1, int occaInnerId2	int _dummy	int dummy
occaFunctionInfo	occaDims, occaInnerId0, occaInnerId1, occaInnerId2	999	1
occaKernel	extern "C"	__kernel	extern "C" __global__
occaFunction			__device__
occaFunctionShared		__local	
occaInnerReturn	continue;	return;	return;

Table A.5 : OCCA keywords related to kernel prototypes and kernel setup.

OCCA	OpenMP	OpenCL	CUDA
occaLocalMemFence		CLK_LOCAL_MEM_FENCE	
occaGlobalMemFence		CLK_GLOBAL_MEM_FENCE	
occaBarrier(Fence)		barrier(Fence)	__syncthreads();

Table A.6 : OCCA barriers needed in parallel threading synchronization.

OCCA	OpenMP	OpenCL	CUDA
occaPrivateArray	occaPrivateClass<type,sz> name	type name[n]	type name[n]
occaPrivate	occaPrivateClass<type,1 > name	type name	type name

Table A.7 : OCCA keywords expanding to platform-dependent private memory types and used to carry over loop-breaks in OpenMP due to barriers.

OCCA	OpenMP	OpenCL	CUDA
occaCPU	1	0	0
occaGPU	0	1	1
occaOpenMP	1	0	0
occaOpenCL	0	1	0
occaCUDA	0	0	1

Table A.8 : OCCA keywords specifying platform for platform-dependent kernel optimization.

APPENDIX B

Appendix: Code

B.1 gNek: OCCA Stiffness Operator Application Kernel

Listing B.1: gNek: OCCA Stiffness Operator Application Kernel

```
occaKernel void semStiffnessOperator(occaKernelInfoArg,
                                     const int occaVariable K,
                                     const datafloat occaVariable lambda,
                                     occaPointer const int       * occaRestrict
                                         galnums,
                                     occaPointer const datafloat * geo,
                                     occaPointer const datafloat * occaRestrict D
                                         ,
                                     occaPointer const datafloat * occaRestrict u
                                         ,
                                     occaPointer       datafloat * occaRestrict
                                         NL){

#if 0
  0<=i,j,k,m<=N AND 0<=k<K

  ur(i,j,k,e) = D(i,m)*u(m,j,k,e)
  us(i,j,k,e) = D(j,m)*u(i,m,k,e)
  ut(i,j,k,e) = D(k,m)*u(i,j,m,e)

  // (grad phi, grad u) = (Dr' Ds' Dt')*(G)*(Dr; Ds; Dt)
   lap(i,j,k,e)  =  D(m,i)*(G(0,m,j,k,e)*ur(m,j,k,e) + G(1,m,j,k,e)*us(m,j,k,e) +
        G(2,m,j,k,e)*ut(m,j,k,e));
```

```
  lap(i,j,k,e) +=  D(m,j)*(G(1,i,m,k,e)*ur(i,m,k,e) + G(3,i,m,k,e)*us(i,m,k,e) +
      G(4,i,m,k,e)*ut(i,m,k,e));
  lap(i,j,k,e) +=  D(m,k)*(G(2,i,j,m,e)*ur(i,j,m,e) + G(4,i,j,m,e)*us(i,j,m,e) +
      G(5,i,j,m,e)*ut(i,j,m,e));

#endif

  occaOuterFor0{

    const int e = occaOuterId0;

    /* advects (u.grad)(u,v,w,T) */
    /* shared register for 'r,s' plane */
    occaShared datafloat LD[Nq][Nq+PAD];
    occaShared datafloat Lu[Nq][Nq+PAD];
    occaShared datafloat Lv[Nq][Nq+PAD];
    occaShared datafloat Lw[Nq][Nq+PAD];

    // u[:][j][i] -> uk[:]
//    occaPrivate(datafloat) uk[Nq];
    occaPrivateArray(datafloat, uk, Nq);

#if REGLAPU==1
//    occaPrivate(datafloat) lapu[Nq];
    occaPrivateArray(datafloat, lapu, Nq);
#else
    occaShared datafloat lapu[Nq][Nq][Nq]  __attribute__ ((__aligned__));;
#endif

    occaInnerFor1{
      occaInnerFor0{
  const unsigned int i = occaInnerId0;
  const unsigned int j = occaInnerId1;

  /* load D into local memory */
  LD[i][j] = D[Nq*j+i]; // D is column major

  /* *** Change load here to use scatter array *** */
  /* load pencil of u into register */
#if UNROLL==1
  occaUnroll
#endif
    for(int k=0;k<Nq;++k) {
      int id = e*BSIZE+k*Nq*Nq+j*Nq+i;
      int gid = galnums[id];

      datafloat tmp = 0.0;

      if(gid >= 0)
        tmp = u[gid]; // this becomes uncoalesced but ** maybe ** cached

      uk[k] = tmp;

//      printf("uk[%d,%d,%d,%d] = %f\n", i,j,k,e, (float)uk[k]);

#if REGLAPU==1
      lapu[k] = 0.f;
#else
      lapu[k][j][i] = 0.f;
#endif
    }
```

```
      }
    }

    occaLocalMemFence;

#if 0
    occaPrivate(datafloat) ut;
    occaPrivate(datafloat) lapuk;
    occaPrivate(datafloat) GDut;
#else
    occaPrivate(datafloat, ut);
    occaPrivate(datafloat, lapuk);
    occaPrivate(datafloat, GDut);
#endif
#if UNROLLOUTER==1
    occaUnroll
#endif
      for(int k=0;k<Nq;++k){

  occaInnerFor1{
    occaInnerFor0{
      const unsigned int i = occaInnerId0;
      const unsigned int j = occaInnerId1;

      Lu[j][i] = uk[k];

      ut = 0.f;

#if UNROLL==1
      occaUnroll
#endif
      for(int m=0;m<Nq;++m) ut += LD[k][m]*uk[m];
    }
  }

  occaLocalMemFence;

  occaInnerFor1{
    occaInnerFor0{
      const unsigned int i = occaInnerId0;
      const unsigned int j = occaInnerId1;

      // prefetch geometric factors
      int id = 7*e*BSIZE+k*Nq*Nq+j*Nq+i;

      const datafloat G00 = geo[id]; id+= BSIZE;
      const datafloat G01 = geo[id]; id+= BSIZE;
      const datafloat G02 = geo[id]; id+= BSIZE;
      const datafloat G11 = geo[id]; id+= BSIZE;
      const datafloat G12 = geo[id]; id+= BSIZE;
      const datafloat G22 = geo[id]; id+= BSIZE;
      const datafloat J   = geo[id];

      datafloat ur = 0, us = 0;

#if UNROLL==1
      occaUnroll
#endif
        for(int m=0;m<Nq;++m) ur += LD[i][m]*Lu[j][m];

#if UNROLL==1
```

```
      occaUnroll
#endif
        for(int m=0;m<Nq;++m) us += LD[j][m]*Lu[m][i];

      Lw[j][i] = G01*ur + G11*us + G12*ut;
      Lv[j][i] = G00*ur + G01*us + G02*ut;

      // put this here for a performance bump
      GDut = G02*ur + G12*us + G22*ut;

      lapuk = J*(lambda*uk[k]);
    }
  }

  occaLocalMemFence;

  occaInnerFor1{
        occaInnerFor0{
      const unsigned int i = occaInnerId0;
      const unsigned int j = occaInnerId1;

#if UNROLL==1
      occaUnroll
#endif
        for(int m=0;m<Nq;++m)
    lapuk += LD[m][j]*Lw[m][i];

#if UNROLL==1
      occaUnroll
#endif

        for(int m=0;m<Nq;++m)
#if REGLAPU==1
    lapu[m] += LD[k][m]*GDut; // DT(m,k)*ut(i,j,k,e)
#else
          lapu[m][j][i] += LD[k][m]*GDut; // DT(m,k)*ut(i,j,k,e)
#endif

#if UNROLL==1
      occaUnroll
#endif
        for(int m=0;m<Nq;++m)
    lapuk+= LD[m][i]*Lv[j][m];

#if REGLAPU==1
      lapu[k] += lapuk;
#else
      lapu[k][j][i] += lapuk;
#endif
    }
  }
      }

#if UNROLL==1
    occaUnroll
#endif
      for(int k=0;k<Nq;++k){
  occaInnerFor1{
    occaInnerFor0{
      const unsigned int i = occaInnerId0;
      const unsigned int j = occaInnerId1;
```

```
      const int id = e*BSIZE+k*Nq*Nq+j*Nq+i;
#if REGLAPU==1
      NL[id] = lapu[k];
#else
      NL[id] = lapu[k][j][i];
#endif
    }
  }
      }
  }
}
```

B.2 RiDG: OCCA Volume Term Application Kernel

Listing B.2: RiDG: OCCA Volume Term Application Kernel

```
#include occaHeader

occaKernel void vtiVolumeKernel(occaKernelInfoArg,
        const int occaVariable K,
        const int occaVariable Klev,
        const datafloat occaVariable alpha,
        occaPointer const int       * occaRestrict  ks,
        occaPointer const datafloat * occaRestrict Dr,
        occaPointer const datafloat * occaRestrict Ds,
        occaPointer const datafloat * occaRestrict Dt,
        occaPointer const datafloat * occaRestrict  geofacs,
        occaPointer const datafloat * occaRestrict  Materials,
        occaPointer const int       * occaRestrict  PTypes,
        occaPointer const datafloat * occaRestrict  ParametersDB,
        occaPointer const datafloat * occaRestrict  Q,
        occaPointer       datafloat * occaRestrict  rhsQ0,
        occaPointer       datafloat * occaRestrict  rhsQ1,
        occaPointer       datafloat * occaRestrict  rhsQ2,
        timefloat occaVariable time){

  occaOuterFor0{

    const int ky = occaOuterId0;

    occaShared datafloat s_sigmaH[p_KblkV][p_Np];
    occaShared datafloat s_sigmaV[p_KblkV][p_Np];
    occaShared datafloat s_u[p_KblkV][p_Np];
    occaShared datafloat s_v[p_KblkV][p_Np];
    occaShared datafloat s_w[p_KblkV][p_Np];

    occaShared datafloat s_geofacs[p_KblkV][9];

    occaPrivate(int, k);
    occaPrivate(int, active);
    occaPrivate(datafloat, sigx);

    int m;
```

```
    occaInnerFor1{
      occaInnerFor0{

  const int n  = occaInnerId0;
  const int kx = occaInnerId1;

  k = kx+ky*p_KblkV;

  active = (k<Klev);

  if(active){
    /// roll history data
    int id = n + k*p_Nfields*p_Npad;

    for(int fld=0;fld<p_Nfields;++fld){
      rhsQ2[id] = rhsQ1[id];
      rhsQ1[id] = rhsQ0[id];
      id += p_Npad;
    }
  }

#define rx s_geofacs[kx][0]
#define ry s_geofacs[kx][1]
#define rz s_geofacs[kx][2]
#define sx s_geofacs[kx][3]
#define sy s_geofacs[kx][4]
#define sz s_geofacs[kx][5]
#define tx s_geofacs[kx][6]
#define ty s_geofacs[kx][7]
#define tz s_geofacs[kx][8]

  if(active){
    if(Klev!=K)
      k = ks[k]-1;

    /// fetch constant geometric factors
    m = 9*k;

    if(n<9)
      s_geofacs[kx][n] = geofacs[m+n];

    /// data parallel read of components of Q at W&B nodes
    int id = n + k*p_Nfields*p_Npad;

    s_sigmaH[kx][n]  = Q[id]; id+= p_Npad;
    s_sigmaV[kx][n]  = Q[id]; id+= p_Npad;
    s_u[kx][n]       = Q[id]; id+= p_Npad;
    s_v[kx][n]       = Q[id]; id+= p_Npad;
    s_w[kx][n]       = Q[id]; id+= p_Npad;

    id = n + k*p_Nmaterials*p_Npad;
    sigx = alpha*Materials[id];

  }
      }
    }

    occaLocalMemFence;

    occaInnerFor1{
```

```
      occaInnerFor0{

  if(active){

    const int n  = occaInnerId0;
    const int kx = occaInnerId1;

    datafloat  dudx = 0,  dvdy = 0, dwdz = 0;
    datafloat  dsigmaHdx = 0,  dsigmaHdy = 0;
    datafloat  dsigmaVdz = 0;
    datafloat dx, dy, dz;
    datafloat Dr_nm, Ds_nm, Dt_nm;

#define bdy           \
    {Dr_nm = Dr[n+m*p_Np];      \
      Ds_nm = Ds[n+m*p_Np];     \
      Dt_nm = Dt[n+m*p_Np];     \
      dx = rx*Dr_nm + sx*Ds_nm + tx*Dt_nm;  \
      dy = ry*Dr_nm + sy*Ds_nm + ty*Dt_nm;  \
      dz = rz*Dr_nm + sz*Ds_nm + tz*Dt_nm;  \
      dsigmaHdx  += dx*s_sigmaH[kx][m];   \
      dsigmaHdy  += dy*s_sigmaH[kx][m];   \
      dsigmaVdz  += dz*s_sigmaV[kx][m];   \
      dudx       += dx*s_u[kx][m];    \
      dvdy       += dy*s_v[kx][m];    \
      dwdz       += dz*s_w[kx][m];    \
      ++m;          \
    }

#include "occa/unroll.hpp"
    for(m=0;m<p_Np;){
      bdy;
    }

    int id = n + k*p_Nfields*p_Npad;
    rhsQ0[id] = dudx + dvdy - sigx*s_sigmaH[kx][n]; id += p_Npad;
    rhsQ0[id] = dwdz        - sigx*s_sigmaV[kx][n];  id += p_Npad;

    /// original rhs [ give or take a factor of rho ]
    rhsQ0[id] = dsigmaHdx - sigx*s_u[kx][n]; id += p_Npad; /// missing rho here
    rhsQ0[id] = dsigmaHdy - sigx*s_v[kx][n]; id += p_Npad;
    rhsQ0[id] = dsigmaVdz - sigx*s_w[kx][n]; id += p_Npad;
  }
      }
    }
  }
}
```

B.3 RiDG: OCCA Surface Term Application Kernel

Listing B.3: RiDG: OCCA Surface Term Application Kernel

```
#include occaHeader
```

```
occaKernel void vtiVolumeKernel(occaKernelInfoArg,
        const int occaVariable K,
        const int occaVariable Klev,
        const datafloat occaVariable alpha,
        occaPointer const int       * occaRestrict  ks,
        occaPointer const datafloat * occaRestrict Dr,
        occaPointer const datafloat * occaRestrict Ds,
        occaPointer const datafloat * occaRestrict Dt,
        occaPointer const datafloat * occaRestrict  geofacs,
        occaPointer const datafloat * occaRestrict  Materials,
        occaPointer const int       * occaRestrict  PTypes,
        occaPointer const datafloat * occaRestrict  ParametersDB,
        occaPointer const datafloat * occaRestrict  Q,
        occaPointer       datafloat * occaRestrict  rhsQ0,
        occaPointer       datafloat * occaRestrict  rhsQ1,
        occaPointer       datafloat * occaRestrict  rhsQ2,
        timefloat occaVariable time){

  occaOuterFor0{

    const int ky = occaOuterId0;

    occaShared datafloat s_sigmaH[p_KblkV][p_Np];
    occaShared datafloat s_sigmaV[p_KblkV][p_Np];
    occaShared datafloat s_u[p_KblkV][p_Np];
    occaShared datafloat s_v[p_KblkV][p_Np];
    occaShared datafloat s_w[p_KblkV][p_Np];

    occaShared datafloat s_geofacs[p_KblkV][9];

    occaPrivate(int, k);
    occaPrivate(int, active);
    occaPrivate(datafloat, sigx);

    int m;

    occaInnerFor1{
      occaInnerFor0{

  const int n  = occaInnerId0;
  const int kx = occaInnerId1;

  k = kx+ky*p_KblkV;

  active = (k<Klev);

  if(active){
    /// roll history data
    int id = n + k*p_Nfields*p_Npad;

    for(int fld=0;fld<p_Nfields;++fld){
      rhsQ2[id] = rhsQ1[id];
      rhsQ1[id] = rhsQ0[id];
      id += p_Npad;
    }
  }

#define rx s_geofacs[kx][0]
#define ry s_geofacs[kx][1]
#define rz s_geofacs[kx][2]
#define sx s_geofacs[kx][3]
```

```
#define sy s_geofacs[kx][4]
#define sz s_geofacs[kx][5]
#define tx s_geofacs[kx][6]
#define ty s_geofacs[kx][7]
#define tz s_geofacs[kx][8]

  if(active){
    if(Klev!=K)
      k = ks[k]-1;

    /// fetch constant geometric factors
    m = 9*k;

    if(n<9)
      s_geofacs[kx][n] = geofacs[m+n];

    /// data parallel read of components of Q at W&B nodes
    int id = n + k*p_Nfields*p_Npad;

    s_sigmaH[kx][n]  = Q[id]; id+= p_Npad;
    s_sigmaV[kx][n]  = Q[id]; id+= p_Npad;
    s_u[kx][n]       = Q[id]; id+= p_Npad;
    s_v[kx][n]       = Q[id]; id+= p_Npad;
    s_w[kx][n]       = Q[id]; id+= p_Npad;

    id = n + k*p_Nmaterials*p_Npad;
    sigx = alpha*Materials[id];

  }
      }
    }

    occaLocalMemFence;

    occaInnerFor1{
      occaInnerFor0{

  if(active){

    const int n  = occaInnerId0;
    const int kx = occaInnerId1;

    datafloat  dudx = 0,  dvdy = 0, dwdz = 0;
    datafloat  dsigmaHdx = 0,  dsigmaHdy = 0;
    datafloat  dsigmaVdz = 0;
    datafloat dx, dy, dz;
    datafloat Dr_nm, Ds_nm, Dt_nm;

#define bdy            \
    {Dr_nm = Dr[n+m*p_Np];      \
      Ds_nm = Ds[n+m*p_Np];     \
      Dt_nm = Dt[n+m*p_Np];     \
      dx = rx*Dr_nm + sx*Ds_nm + tx*Dt_nm;  \
      dy = ry*Dr_nm + sy*Ds_nm + ty*Dt_nm;  \
      dz = rz*Dr_nm + sz*Ds_nm + tz*Dt_nm;  \
      dsigmaHdx  += dx*s_sigmaH[kx][m];   \
      dsigmaHdy  += dy*s_sigmaH[kx][m];   \
      dsigmaVdz  += dz*s_sigmaV[kx][m];   \
      dudx       += dx*s_u[kx][m];    \
      dvdy       += dy*s_v[kx][m];    \
      dwdz       += dz*s_w[kx][m];    \
```

```
      ++m;           \
    }

#include "occa/unroll.hpp"
    for(m=0;m<p_Np;){
      bdy;
    }

    int id = n + k*p_Nfields*p_Npad;
    rhsQ0[id] = dudx + dvdy - sigx*s_sigmaH[kx][n]; id += p_Npad;
    rhsQ0[id] = dwdz        - sigx*s_sigmaV[kx][n];  id += p_Npad;

    /// original rhs [ give or take a factor of rho ]
    rhsQ0[id] = dsigmaHdx - sigx*s_u[kx][n]; id += p_Npad; /// missing rho here
    rhsQ0[id] = dsigmaHdy - sigx*s_v[kx][n]; id += p_Npad;
    rhsQ0[id] = dsigmaVdz - sigx*s_w[kx][n]; id += p_Npad;
  }
      }
    }
  }
}
```

References

[1] TOP500 Supercomputer Site.

[2] Akhtar Ali, Usman Dastgeer, and Christoph Kessler. Opencl for programming shared memory multicore cpus. In *MULTIPROG-2012 Workshop at HiPEAC-2012*, 2012.

[3] Christopher G Baker, Michael A Heroux, H Carter Edwards, and Alan B Williams. A light-weight api for portable multicore programming. In *Parallel, Distributed and Network-Based Processing (PDP), 2010 18th Euromicro International Conference on*, pages 601–606. IEEE, 2010.

[4] Donald J Becker, Thomas Sterling, Daniel Savarese, John E Dorband, Udaya A Ranawak, and Charles V Packer. Beowulf: A parallel workstation for scientific computation. In *Proceedings, International Conference on Parallel Processing*, volume 95, 1995.

[5] James L Black and Matthew A Brzostowski. Systematics of time-migration errors. *Geophysics*, 59(9):1419–1434, 1994.

[6] Francois Bodin and Stephane Bihan. Heterogeneous multicore parallel programming for graphics processing units. *Scientific Programming*, 17(4):325–336, 2009.

[7] Tobias Brandvik and Graham Pullan. An accelerated 3d navier–stokes solver for flows in turbomachines. *Journal of Turbomachinery*, 133(2):021025, 2011.

[8] Ian Buck, Tim Foley, Daniel Horn, Jeremy Sugerman, Kayvon Fatahalian, Mike Houston, and Pat Hanrahan. Brook for gpus: stream computing on graphics hardware. In *ACM Transactions on Graphics (TOG)*, volume 23, pages 777–786. ACM, 2004.

[9] Allan Campbell. Ibm have not stopped cell processor development, November 2009.

[10] W Dally. Throughput computing. Keynote at International Conference on Supercomputing, 2010.

[11] Robert H Dennard, Fritz H Gaensslen, V Leo Rideout, Ernest Bassous, and Andre R LeBlanc. Design of ion-implanted mosfet's with very small physical dimensions. *Solid-State Circuits, IEEE Journal of*, 9(5):256–268, 1974.

[12] Advanced Micro Devices. Porting cuda applications to opencl.

[13] Gregory Frederick Diamos, Andrew Robert Kerr, Sudhakar Yalamanchili, and Nathan Clark. Ocelot: a dynamic optimization framework for bulk-synchronous applications in heterogeneous systems. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, pages 353–364. ACM, 2010.

[14] Romain Dolbeau, Stéphane Bihan, and François Bodin. Hmpp: A hybrid multi-core parallel programming environment. In *Workshop on General Purpose Processing on Graphics Processing Units (GPGPU 2007)*, 2007.

[15] Eric Duveneck, Paul Milcik, Peter M Bakker, and Colin Perkins. Acoustic vti wave equations and their application for anisotropic reverse-time migration. In *SEG Technical Program Expanded Abstracts 2008*, pages 2186–2190. Society of Exploration Geophysicists, 2008.

[16] Hadi Esmaeilzadeh, Emily Blem, Renee St Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *Computer Architecture (ISCA), 2011 38th Annual International Symposium on*, pages 365–376. IEEE, 2011.

[17] P Fischer, J Kruse, J Mullen, H Tufo, J Lottes, and S Kerkemeier. Nek5000–open source spectral element cfd solver. *Argonne National Laboratory, Mathematics and Computer Science Division, Argonne, IL, see https://nek5000. mcs. anl. gov/index. php/MainPage*, 2008.

[18] Paul F Fischer and Anthony T Patera. Parallel spectral element solution of the stokes problem. *Journal of Computational Physics*, 92(2):380–421, 1991.

[19] Paul F Fischer and Einar M Rωnquist. Spectral element methods for large scale parallel navierstokes calculations. *Computer methods in applied mechanics and engineering*, 116(1):69–76, 1994.

[20] Rajesh Gandham, Ken Esler, and Yongpeng Zhang. A GPU accelerated aggregation algebraic multigrid method. *Computers and Mathematics with Applications*, submitted.

[21] Rajesh Gandham, David Medina, and T. Warburton. GPU accelerated numerical methods for tsunami modeling (poster), 2013.

[22] Rajesh Gandham, David Medina, and T. Warburton. Gpu accelerated discontinuous galerkin methods for shallow water equations. *Communications in Computational Physics(In submission)*, (In submission).

[23] Rajesh Gandham and T. Warburton. ALMOND: Algrebraic multigrid on numerous devices (poster), 2014.

[24] Dominik Goddeke, Sven HM Buijssen, Hilmar Wobker, and Stefan Turek. Gpu acceleration of an unmodified parallel finite element navier-stokes solver. In *High Performance Computing & Simulation, 2009. HPCS'09. International Conference on*, pages 12–21. IEEE, 2009.

[25] Dominik Göddeke, Robert Strzodka, and Stefan Turek. *Accelerating double precision FEM simulations with GPUs*. Univ. Dortmund, Fachbereich Mathematik, 2005.

[26] JL Guermond, P. Minev, and J. Shen. An overview of projection methods for incompressible flows. *Computer Methods in Applied Mechanics and Engineering*, 195(44):6011–6045, 2006.

[27] CW Hamman, Robert M Kirby, and Martin Berzins. Parallelization and scalability of a spectral element channel flow solver for incompressible navier–stokes equations. *Concurrency and Computation: Practice and Experience*, 19(10):1403–1422, 2007.

[28] Mark Harris et al. Optimizing parallel reduction in cuda. *NVIDIA Developer Technology*, 2:45, 2007.

[29] Matt J Harvey and Gianni De Fabritiis. Swan: A tool for porting cuda programs to opencl. *Computer Physics Communications*, 182(4):1093–1099, 2011.

[30] Jan S Hesthaven and Tim Warburton. Nodal discontinuous galerkin methods: Algorithms. *Analysis, and Applications*, 54:95, 2008.

[31] Intel. Embedded pentium©processors with mmx™technology.

[32] G.E. Karniadakis, M. Israeli, and S.A. Orszag. High-order splitting methods for the incompressible navier-stokes equations. *Journal of computational physics*, 97(2):414–443, 1991.

[33] Victor W Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, et al. Debunking the 100x gpu vs. cpu myth:

an evaluation of throughput computing on cpu and gpu. In *ACM SIGARCH Computer Architecture News*, volume 38, pages 451–460. ACM, 2010.

[34] Ami Marowka. Performance study of the first three intel multicore processors. *Scalable Computing: Practice and Experience*, 10(4), 2001.

[35] Gabriel Martinez, Mark Gardner, and Wu-chun Feng. Cu2cl: A cuda-to-opencl translator for multi-and many-core architectures. In *Parallel and Distributed Systems (ICPADS), 2011 IEEE 17th International Conference on*, pages 300–307. IEEE, 2011.

[36] David Medina, Amik St. Cyr, and T. Warburton. OCCA: A unified approach to multi-threading languages (poster), 2014.

[37] Paulius Micikevicius. 3d finite difference computation on gpus using cuda. In *Proceedings of 2nd Workshop on General Purpose Processing on Graphics Processing Units*, pages 79–84. ACM, 2009.

[38] Paulius Micikevicius. Gpu performance analysis and optimization. In *GPU Technology Conference, http://developer. download. nvidia. com/GTC/PDF/GTC2012/PresentationPDF/S0514-GTC2012-GPU-Performance-Analysis. pdf*, 2012.

[39] Tim Moon, Michael Franco, Jesse Chan, and Tim Warburton. brainNek: Real-time laser-induced thermal therapy simulation. *Journal (Not submitted yet)*, Year (Not submitted yet).

[40] Aaftab Munshi. Opencl: Parallel computing on the gpu and cpu. *SIGGRAPH, Tutorial*, 2008.

[41] Karl Rupp, Florian Rudolf, and Josef Weinbub. Viennacl-a high level linear algebra library for gpus and multi-core cpus. *Proc. GPUScA*, pages 51–56, 2010.

[42] Robert R Schaller. Moore's law: past, present and future. *Spectrum, IEEE*, 34(6):52–59, 1997.

[43] Julien C Thibault and Inanc Senocak. Cuda implementation of a navier-stokes solver on multi-gpu desktop platforms for incompressible flows. In *Proceedings of the 47th AIAA Aerospace Sciences Meeting*, pages 2009–758, 2009.

[44] Leon Thomsen. Weak elastic anisotropy. *Geophysics*, 51(10):1954–1966, 1986.

[45] Henry M Tufo and Paul F Fischer. Terascale spectral element algorithms and implementations. In *Proceedings of the 1999 ACM/IEEE conference on Supercomputing (CDROM)*, page 68. ACM, 1999.

[46] Freddie Witherden, Antony Farrington, and Peter Vincent. Pyfr: An open source python framework for high-order cfd on many-core platforms. *FEMTEC*, page 141, 2013.

[47] Yu Zhang, James Sun, Samuel Gray, et al. Reverse-time migration: amplitude and implementation issues. In *2007 SEG Annual Meeting*. Society of Exploration Geophysicists, 2007.